Models and Algorithms

Regression and Dimensionality Reduction

Overview

★ Supervised learning - Regression Models:

Linear regression, NNs for regression, Gaussian Process Regression

★ Unsupervised Learning – Dimensionality Reduction

PCA, ICA, AutoEncoder

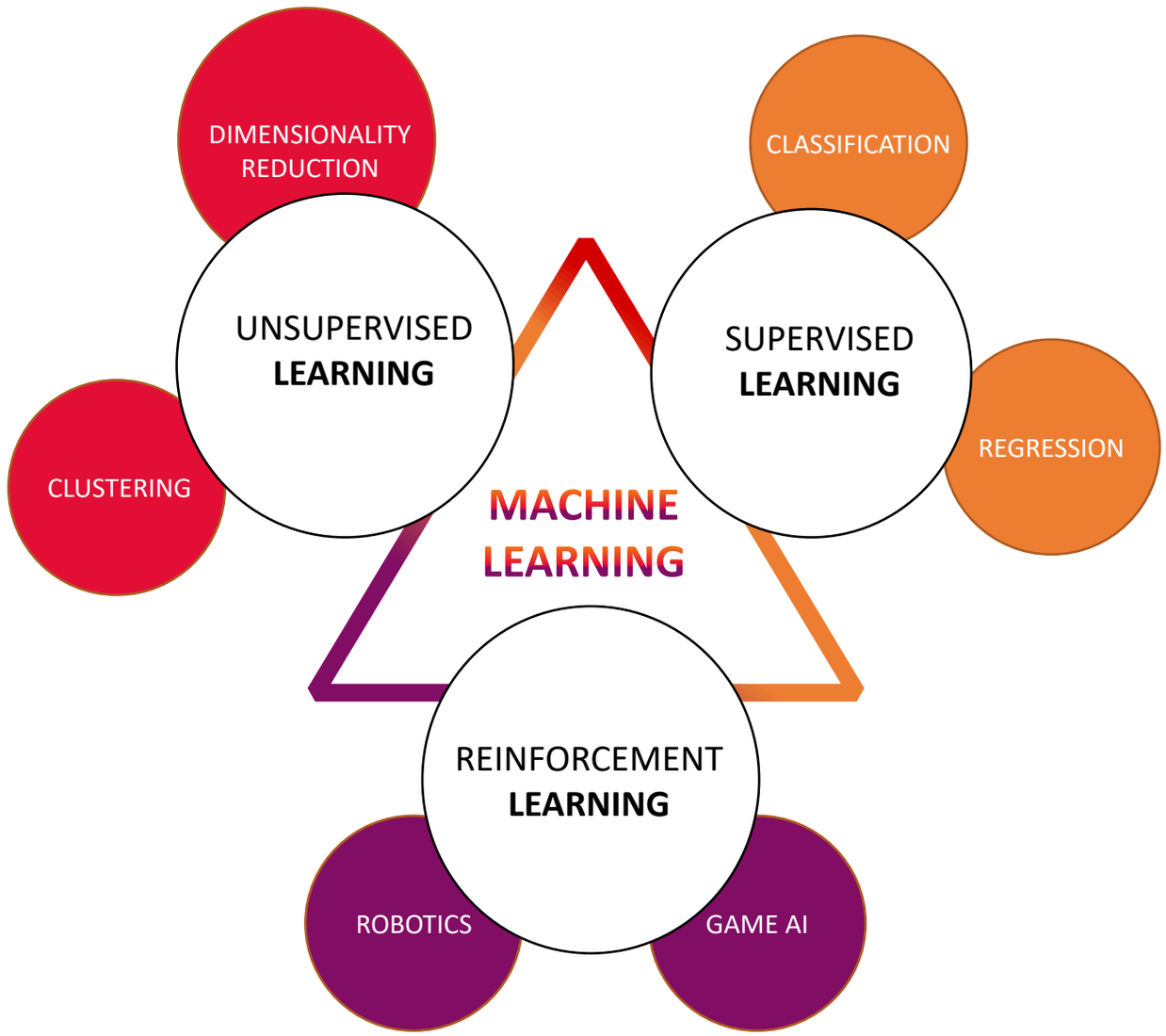
★ Unsupervised Learning – Clustering Models

K-Means, Gaussian mixture models, Affinity Propagation,

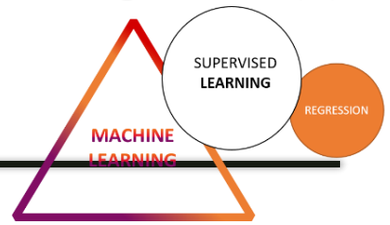
Agglomerative Clustering, DBSCAN

★ Model Selection

Regression Methods



Regression Analysis



Produce a regression equation

- Regression analysis is widely used for prediction

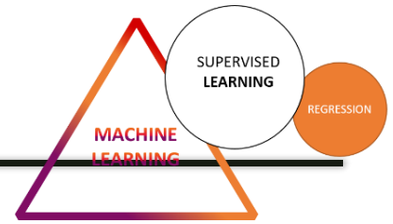
- Describe the relationships between a set of independent variables and the dependent variable

- Describe how the changes in each independent variable (X_i) are related to changes in the dependent variable (Y)

- Difference between Regression and Classification?

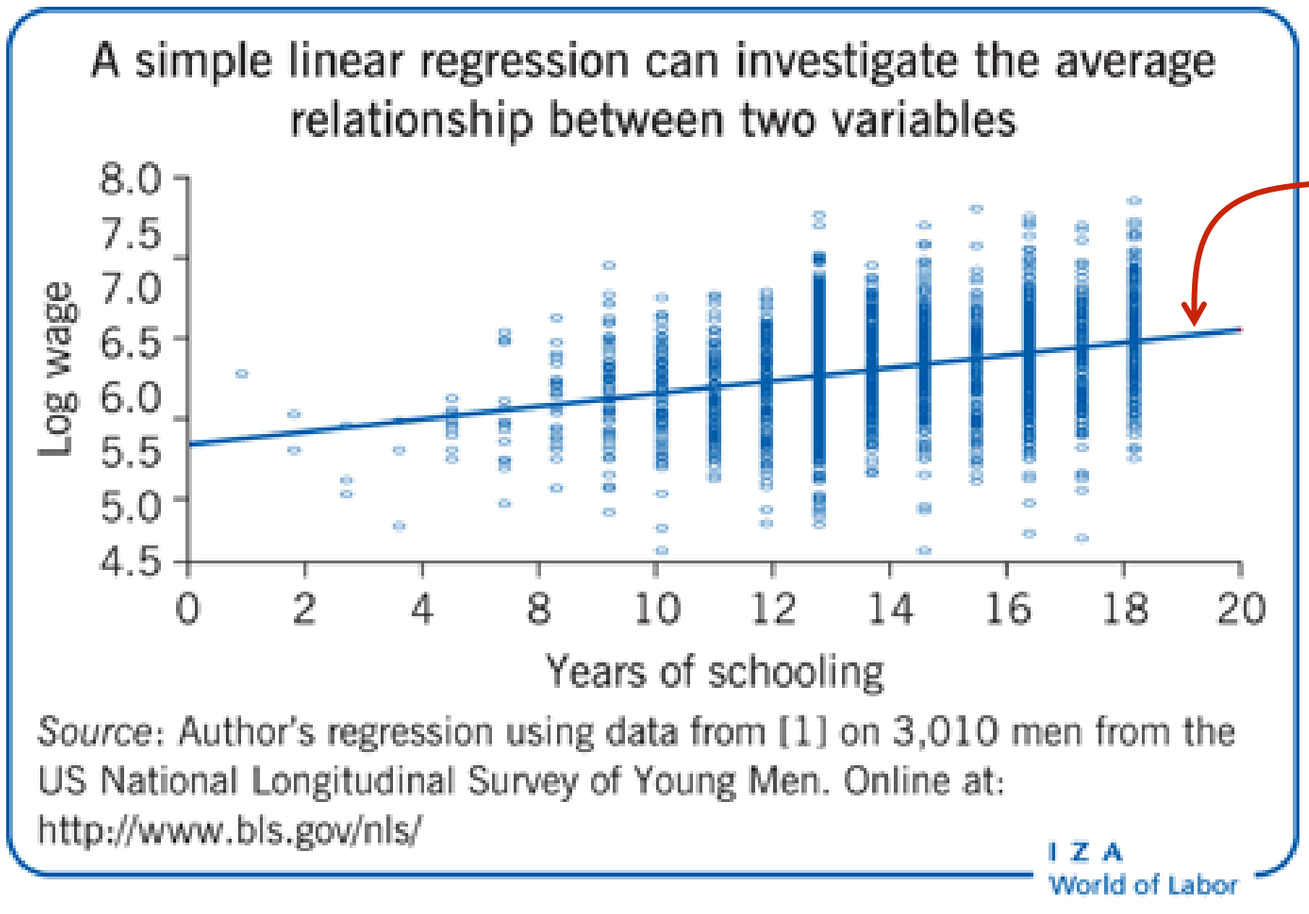
Output: A continuous quantify output vs. A discrete class label

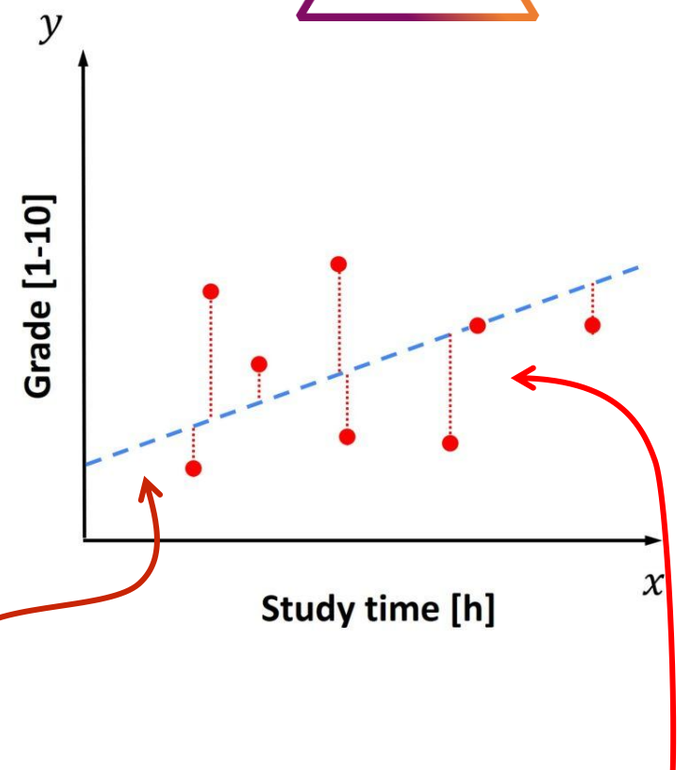
Simple Linear Regression



- Linear Regression

$$y_i = \alpha + \beta x_i + \varepsilon_i$$



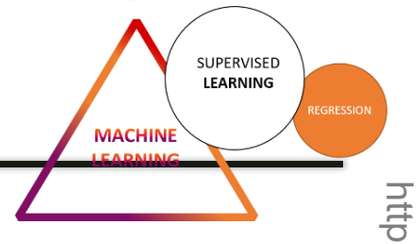


learn slopes and intercepts:

- start with random values
- "Loss function" or "Cost": the **sum** of **squared** errors
- we use an *optimiser* to find values that *minimise* this loss function

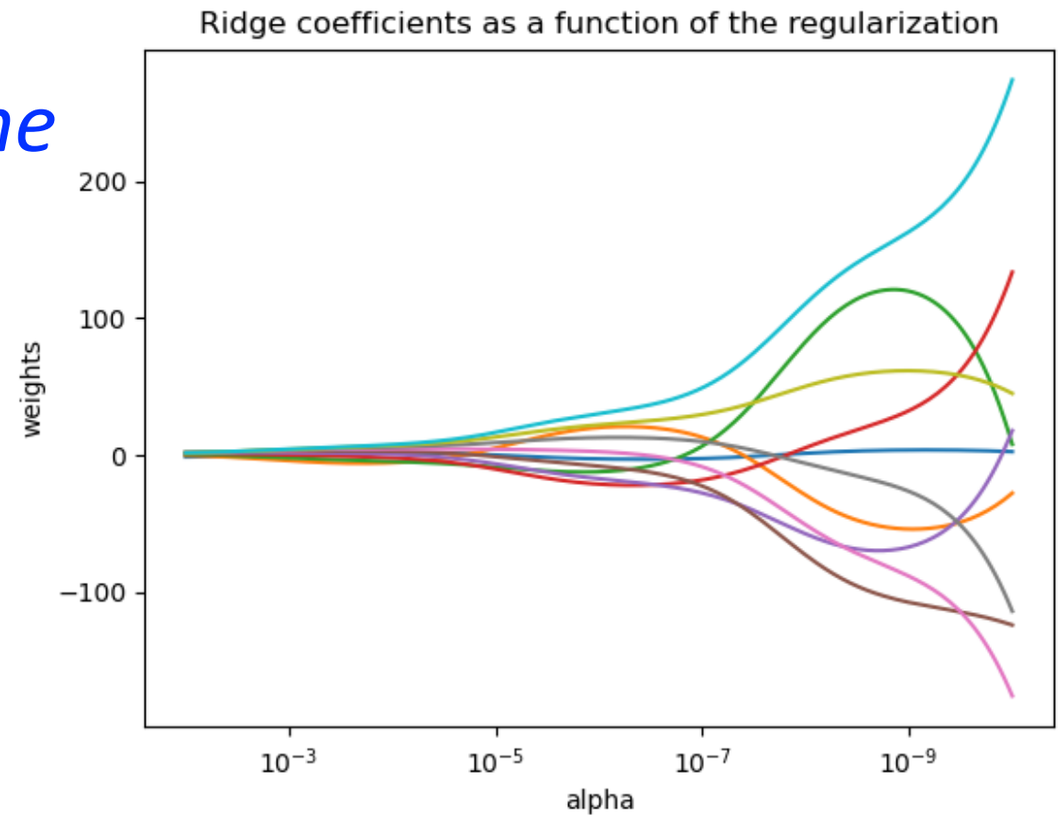
- Simple but useful, good interpretability

Multiple Linear Regression



- in 1-d: fit a *straight line*…
- in more dimensions: fit a hyper*plane*
- one intercept, but many slopes, usually called ***coefficients/weights***

$$y_i = \alpha + \sum_{k=0}^{m} \beta_k x_{k,i}$$



https://scikit-learn.org/stable/auto_examples/linear_model/

Regularisation?

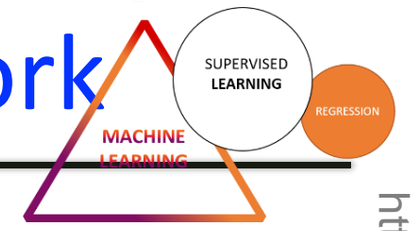
add a penalty to the cost function

- sum of squared weights
  a.k.a. "L2 regularisation"/"ridge regression"
  scikit-learn sets the penalty via "alpha"
- sum of abs() of weights
  a.k.a. "L1 regularisation"/"lasso regression"

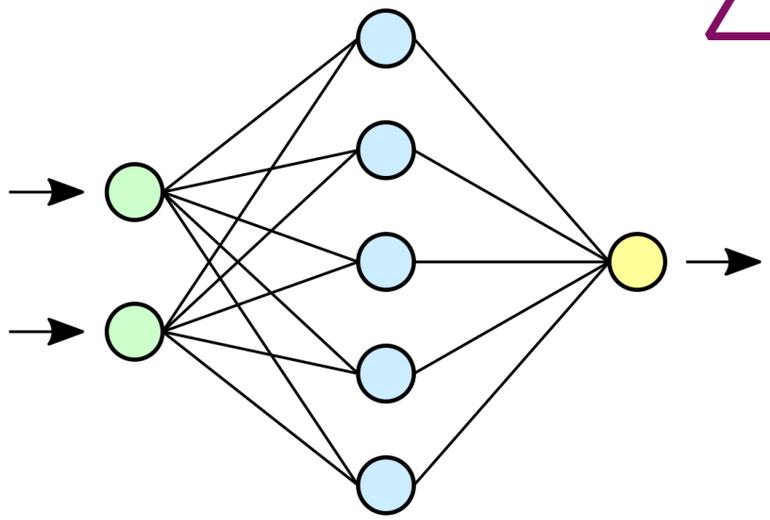
- "Loss function" or "Cost":
  the **sum** of **squared** errors

  + alpha times
  the sum of squared weights

Non-linear Regression with a Neural Network

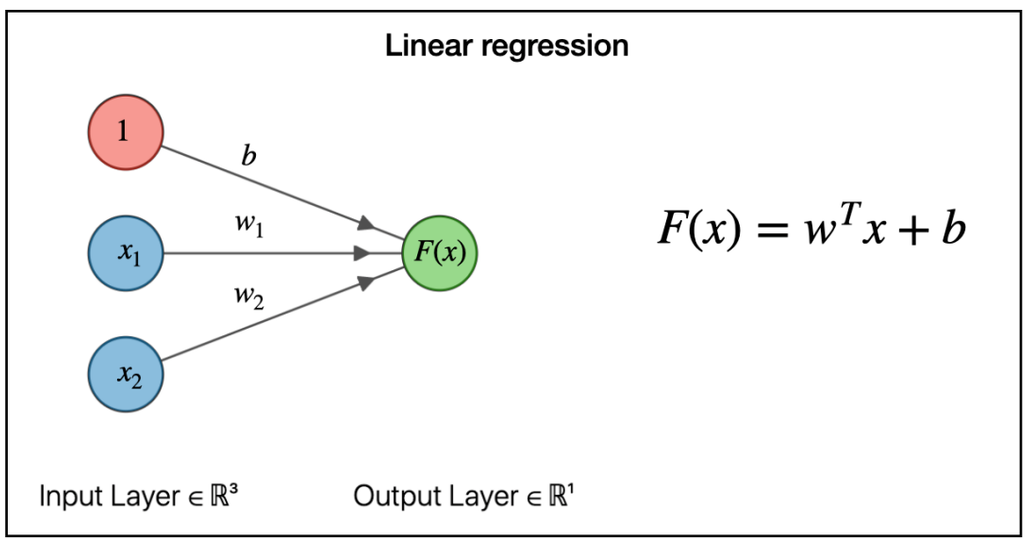


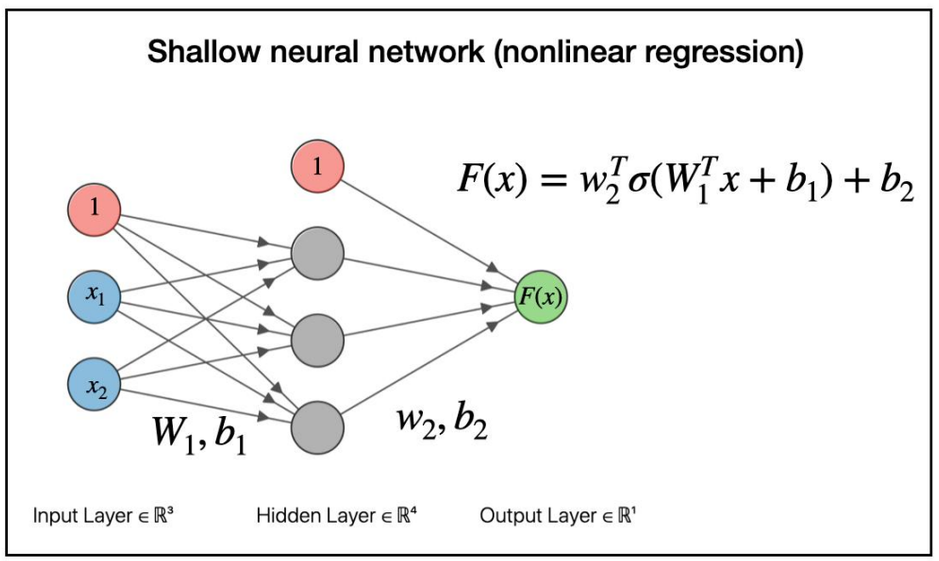
Linear regression:
map green directly to yellow,
linearly, with weights



Non-linear: 1st map green to blue (with anything, maybe weights/neurons)
then map blue to yellow
linearly, with weights

"Loss function" or "Cost":
the **sum** of **squared** errors
+ alpha times
the sum of squared weights

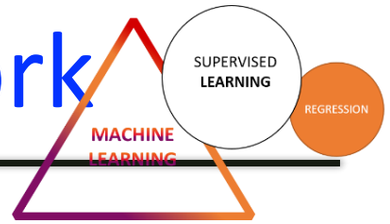




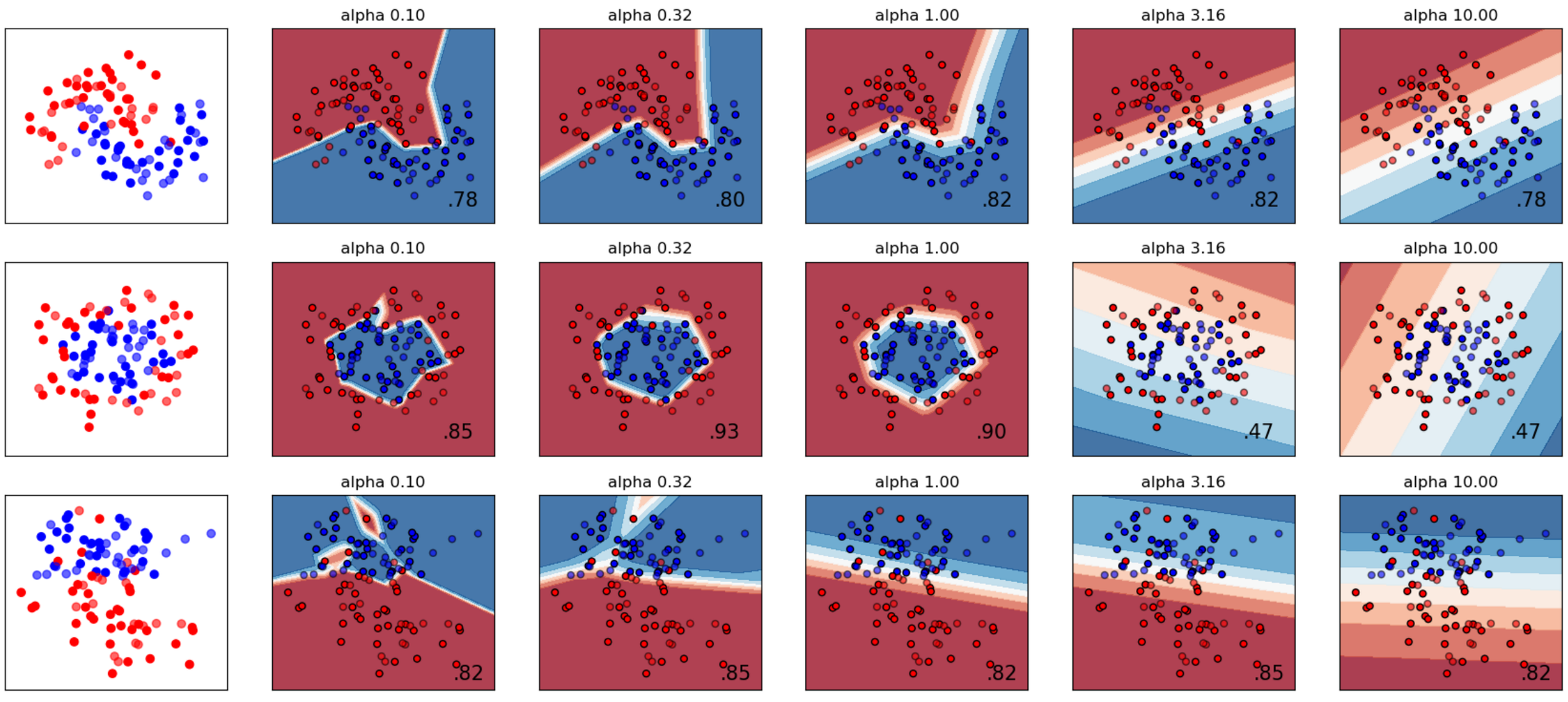
$$F(x) = w_1 \cdot x_1 + w_2 \cdot x_2 + 1 \cdot b$$

https://scikit-learn.org/stable/modules/neural_networks_supervised.html

Non-linear Regression with a Neural Network

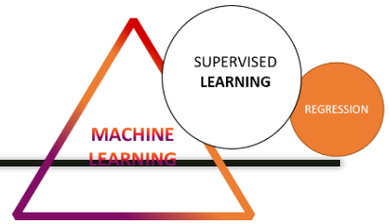


- *"alpha"* for regularization: penalising large weights, avoid overfitting

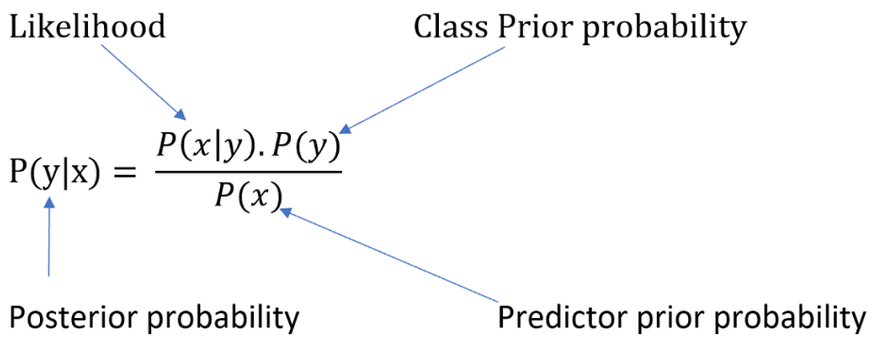


https://scikit-learn.org/stable/modules/neural_networks_supervised.html

Gaussian Process Regression



- A Bayesian approach for regression



- A Gaussian processes model describes a probability distribution over possible functions that fit a set of points

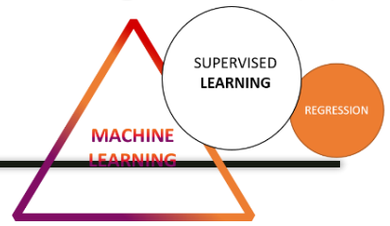
- Regression model given by a multivariate Gaussian

$$P(f|X) = \mathcal{N}(f|\mu, K)$$

μ are the mean functions, ***K*** is the Kernel function for covariance

- Gaussian processes model: a predictive distribution over functions whose shape is defined by ***K***

Gaussian Process Regression

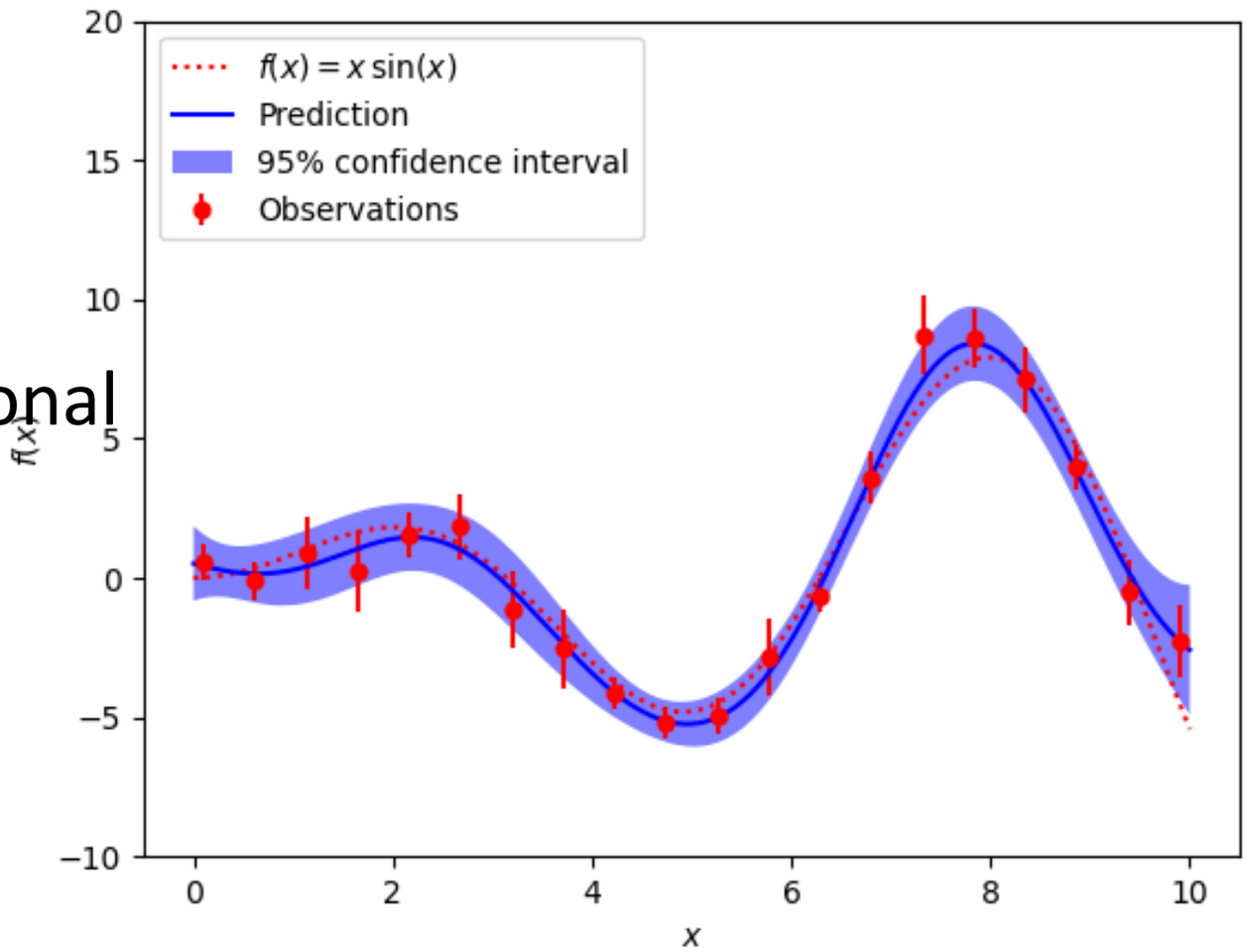


a *very* different way of thinking…!

- just a few parameters to learn from data (e.g. 3)
- Errorbars ☺ on predictions
- predictions depend *directly* on previous data
- lose efficiency in high dimensional spaces

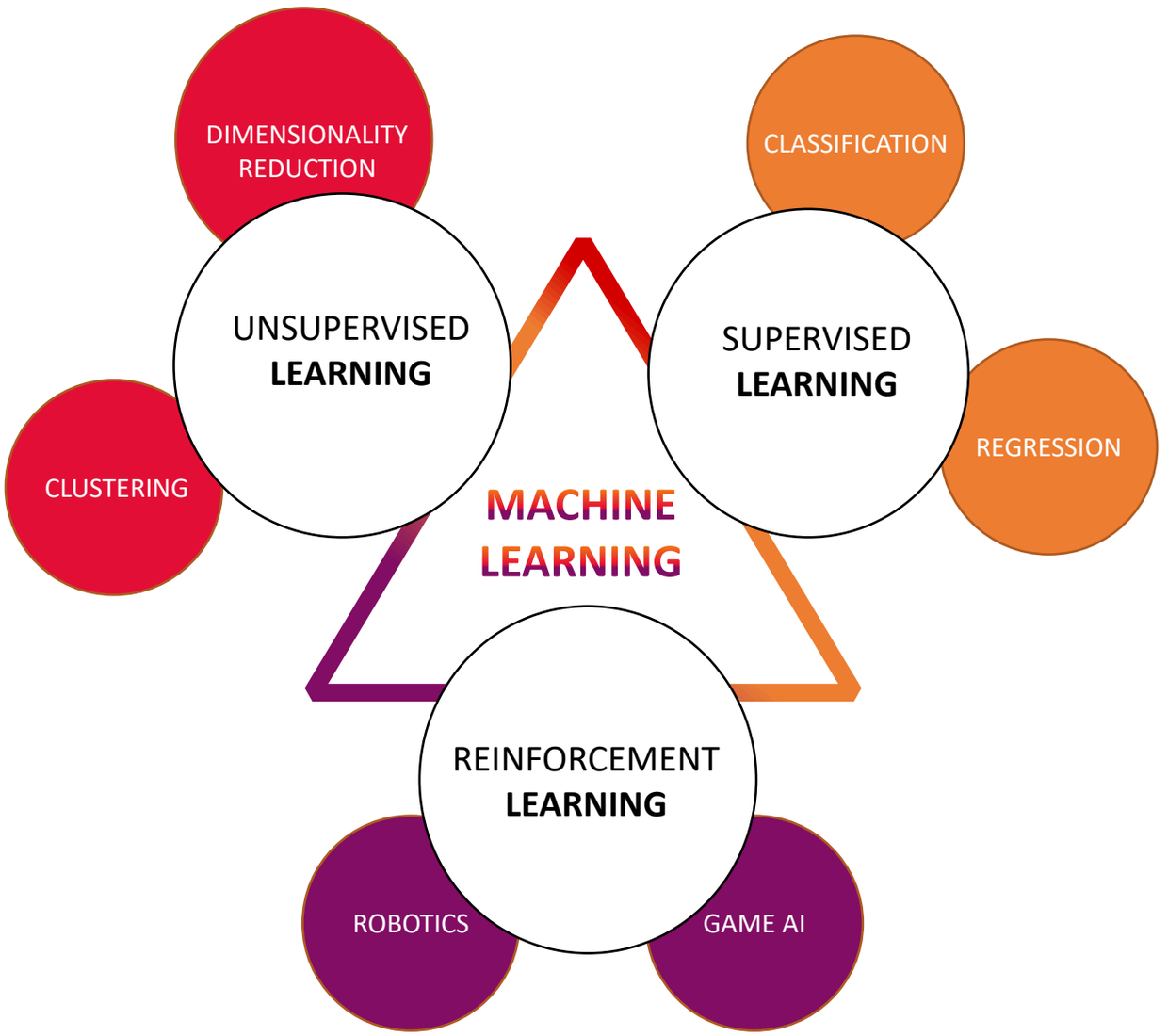
cf a large neural network:

- millions of parameters
- no errorbars
- predictions depend on parameters, but not *directly* on past data

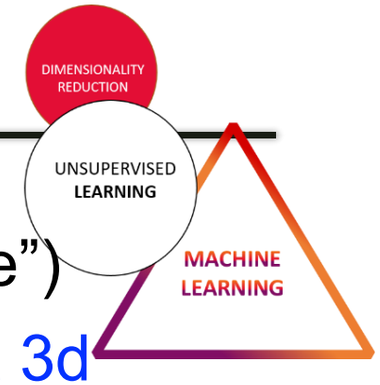


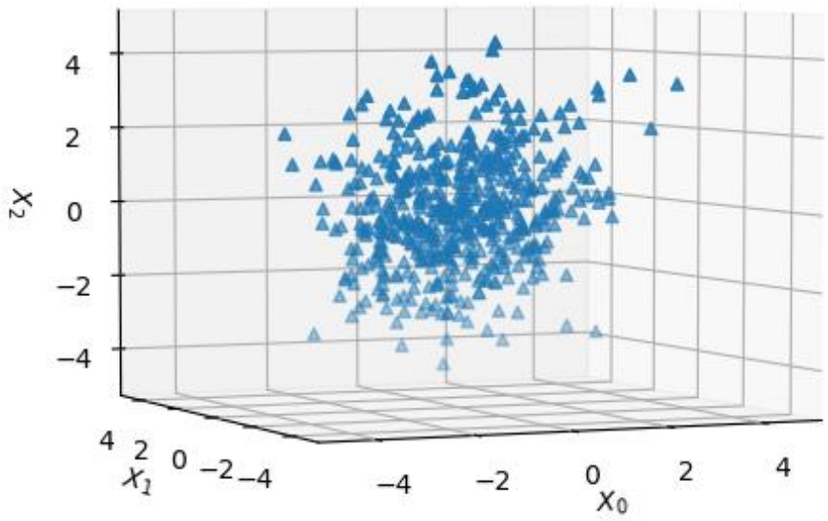
https://scikit-learn.org/0.24/auto_examples/gaussian_process/plot_gpr_noisy_targets.html

Dimensionality Reduction



Dimensionality Reduction





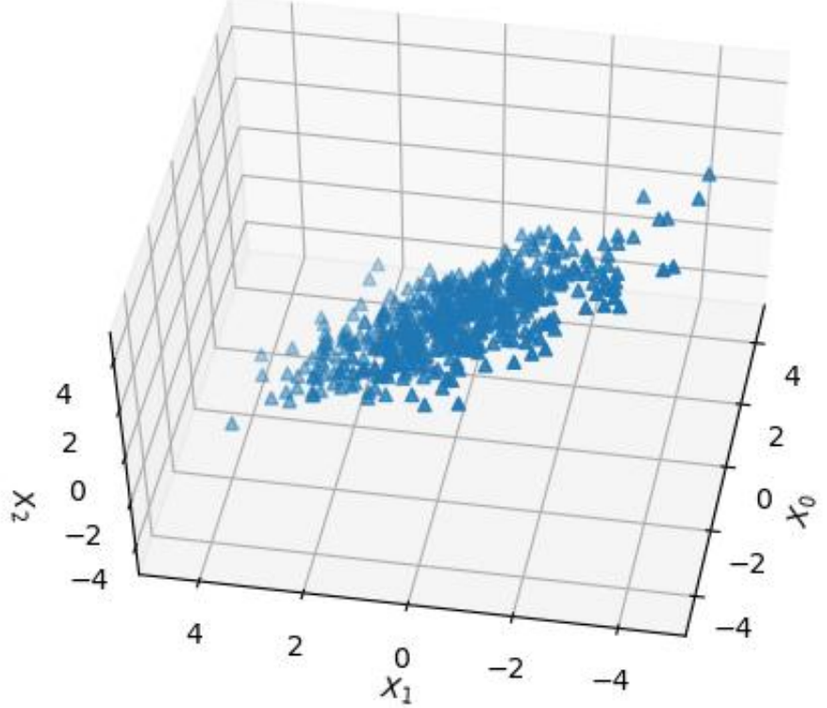
Data might exist on a manifold (a “subspace”)

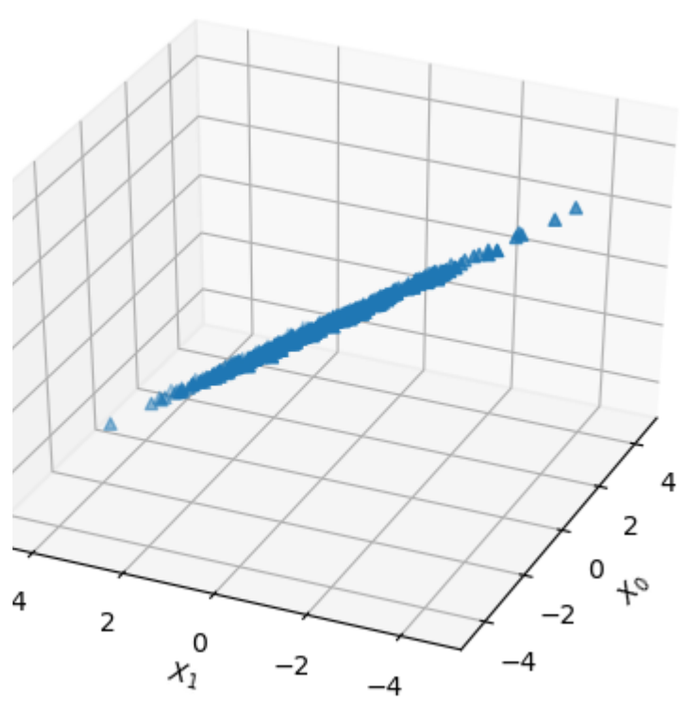
e.g. this data sits on a 2d “plain”, sitting in a 3d feature space

“dimensionality reduction” techniques map each point into a low-dim space

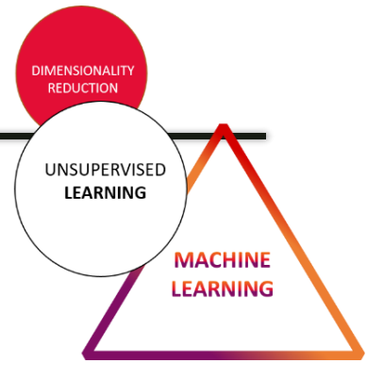
What’s the point?

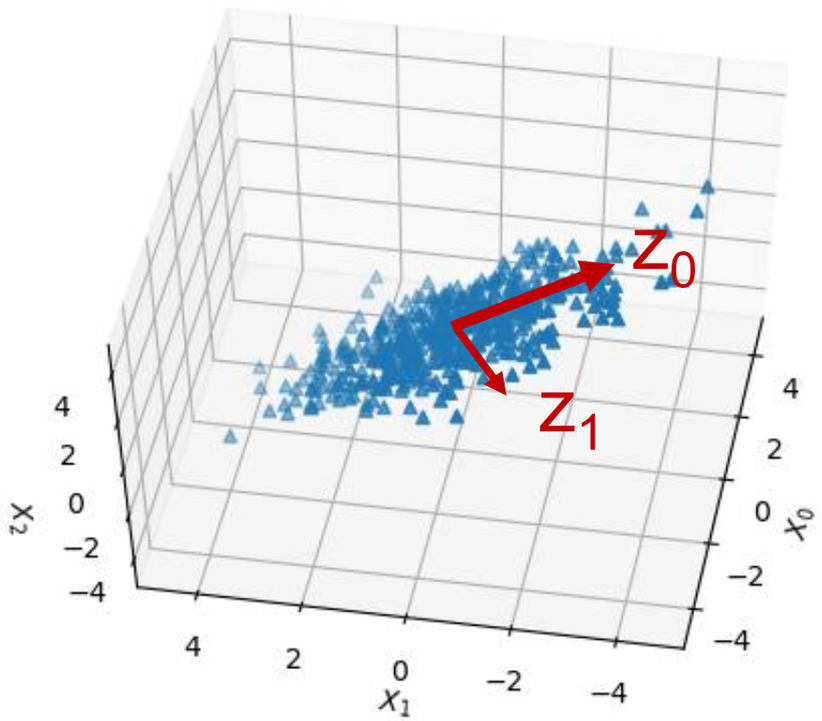
1. insight: the “real” underlying factors
2. can be useful pre-processor of data, e.g. feed a classifier the low-dim version, to avoid over-fitting





PCA: principal components analysis

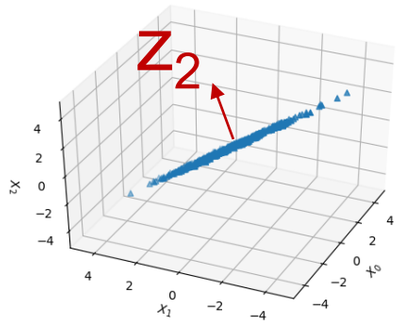




If the manifold is linear (a plain), we can easily find it: an algorithm known as PCA

"rotates space" (x_0, x_1, x_2) → (z_0, z_1, z_2) such that :

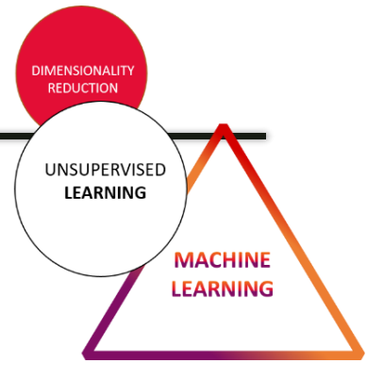
- z_0 is the direction of most variation, called the "1st principal component",
- z_1 is the direction of next-to-most variation, called "2nd principal component",
- and so on…



If z_2 has very little variation (as here), we might just ***drop it***, which means we've reduced the dimensionality of the data, in effect:

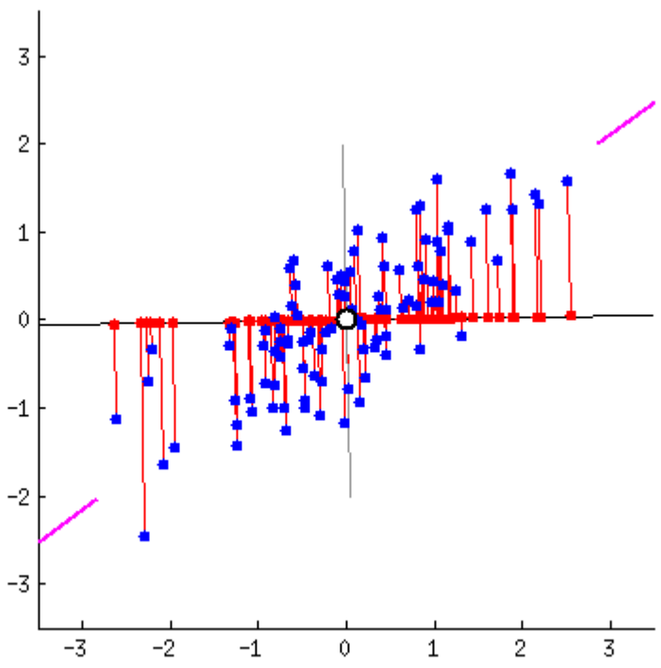
(x_0, x_1, x_2) → (z_0, z_1)

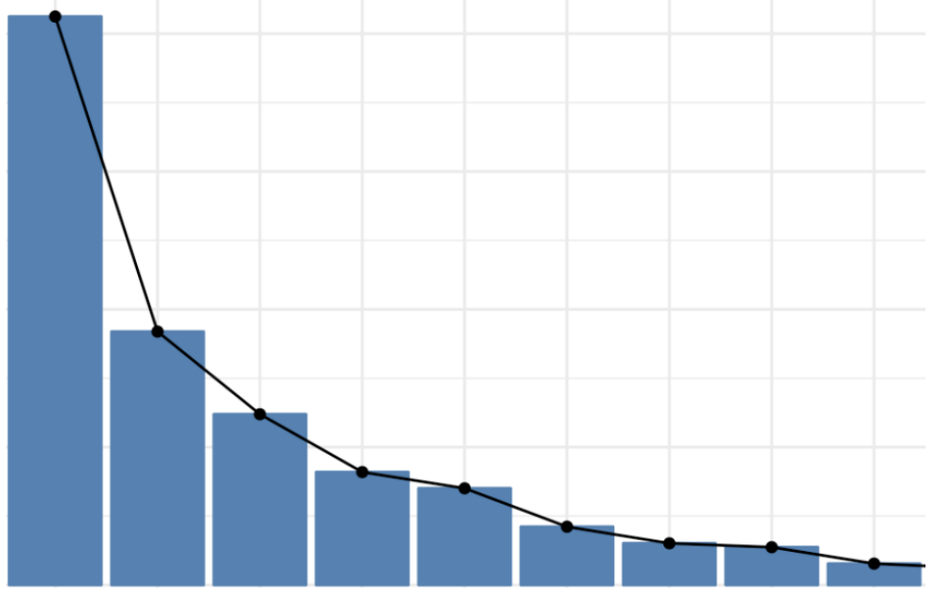
PCA: principal components analysis



- use an orthogonal transformation to convert a set of observations with possibly correlated variables into a set of values of (linearly) uncorrelated variables

- Principal components: new variables that are constructed as linear combinations or mixtures of the initial variables

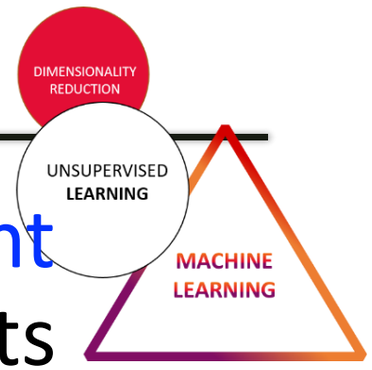




Variance Capture by Principal Components

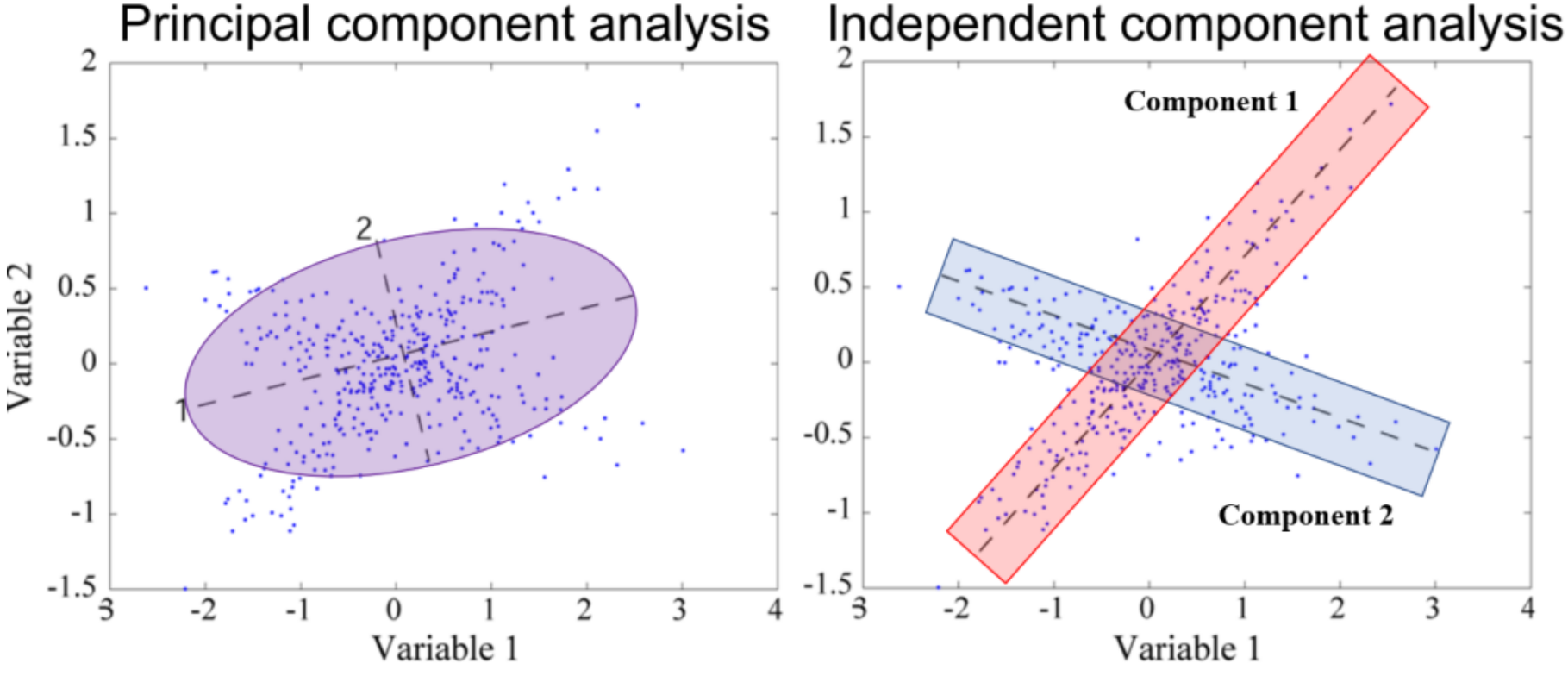
https://builtin.com/data-science/step-step-explanation-principal-component-analysis

ICA: independent components analysis



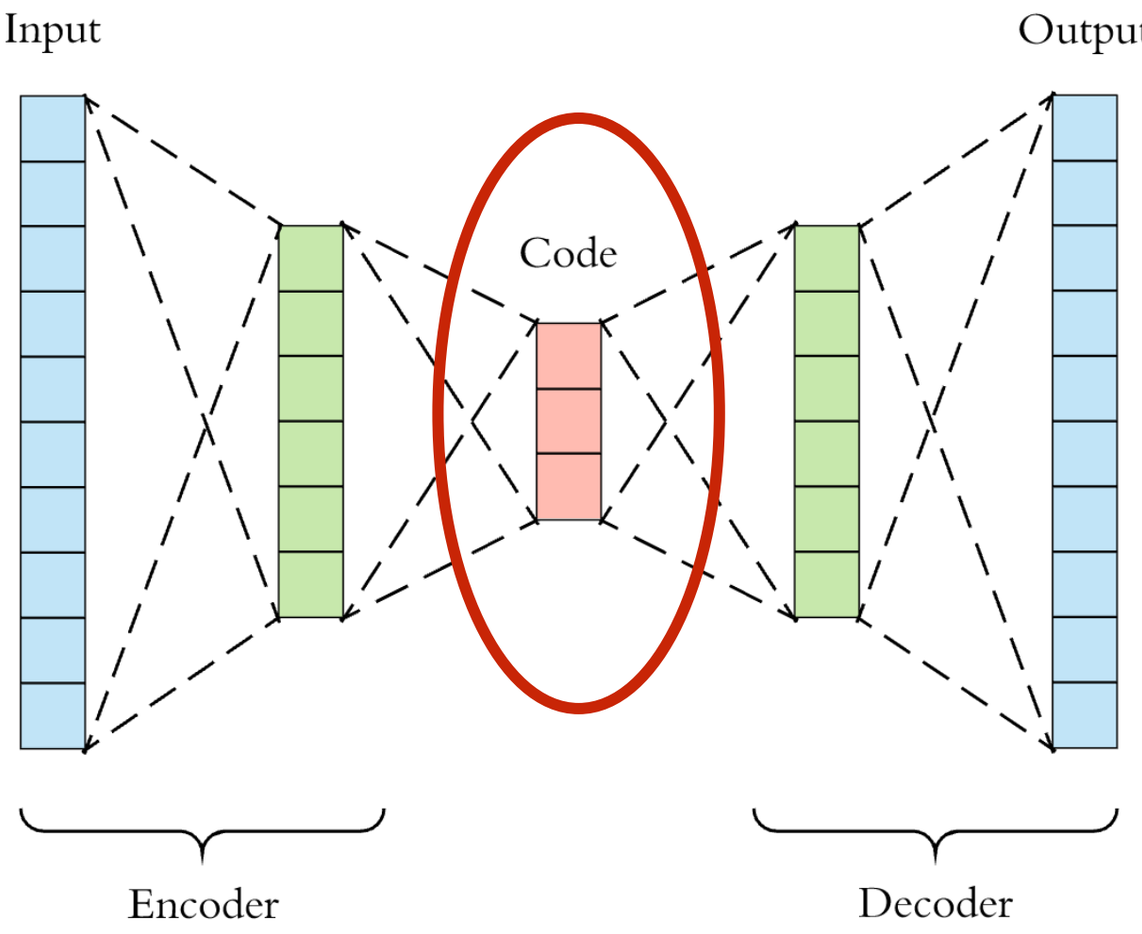
- assume each data variable is a mixture of independent components and aim to find independent components

- output a source matrix with columns as independent sources



Autoencoders

- Autoencoders – a special type of neural network
- train to copy its input to output
- a “thin” layer – hidden layer that describe a code used to represent the input
- minimise the reconstruction Loss



Lectore 06 – Models and Algorithms

Clustering

Overview

★ Supervised learning - Regression Models:

Linear regression, NNs for regression, Gaussian Process Regression

★ Unsupervised Learning – Dimension Reduction Methods

PCA, ICA, AutoEncoder

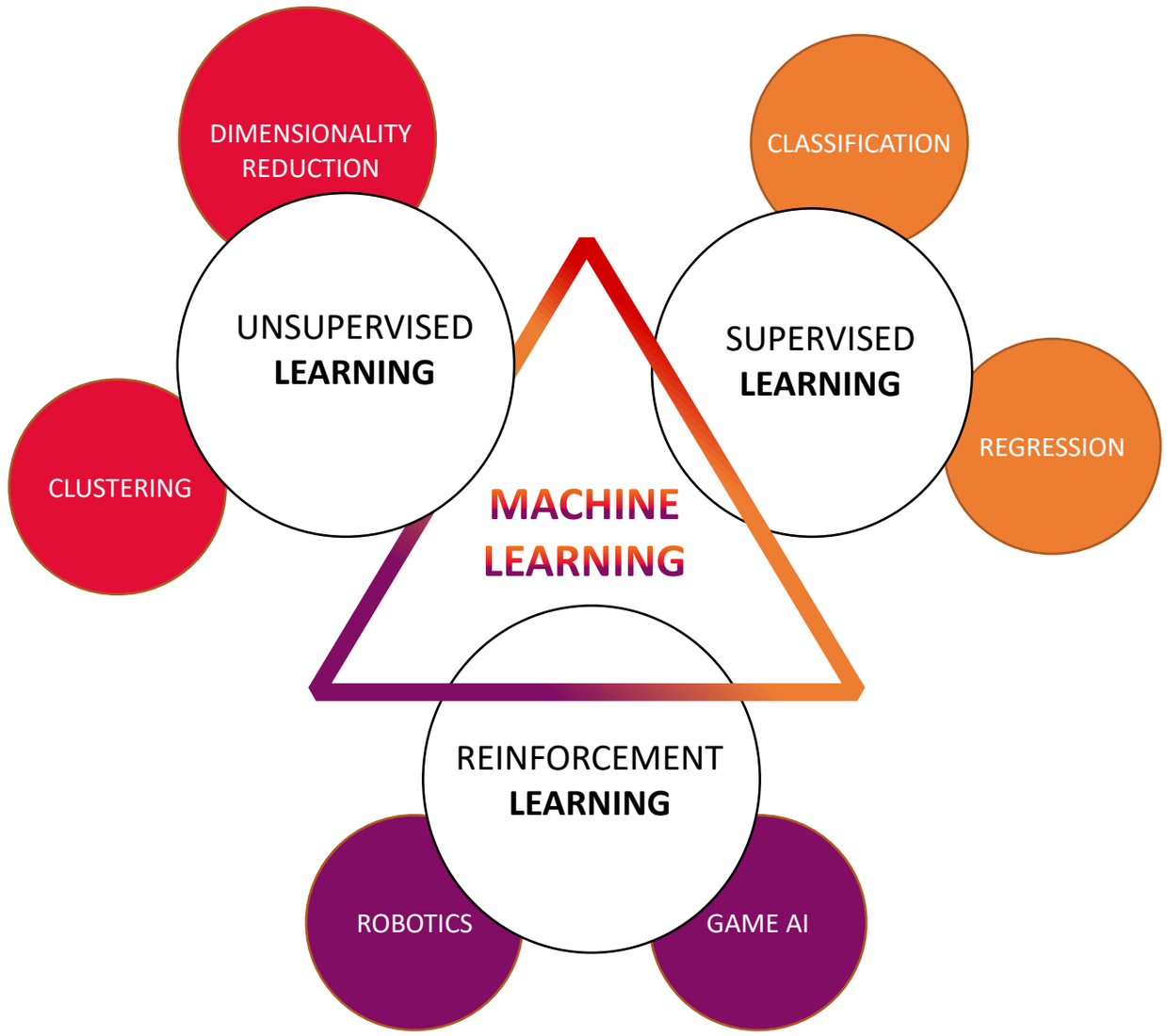
★ Unsupervised Learning – Clustering Models

K-Means, Gaussian mixture models, Affinity Propagation,

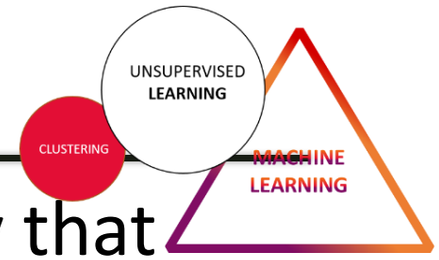
Agglomerative Clustering, DBSCAN, Mean Shift, BIRCH

★ Model Selection

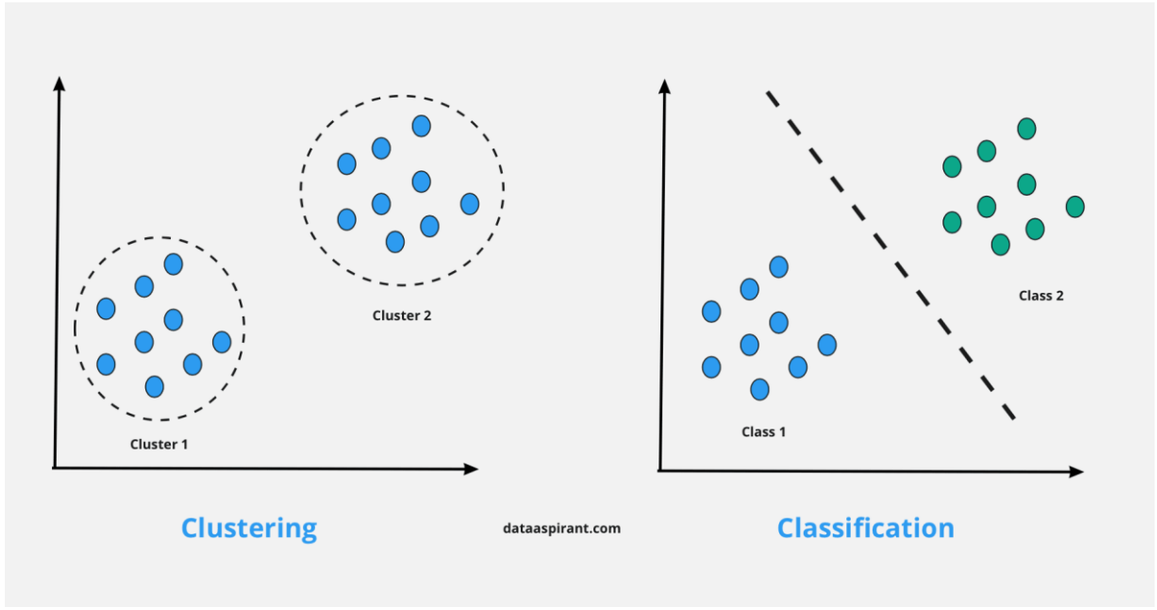
Tasks


DIMENSIONALITY REDUCTION
CLUSTERING
UNSUPERVISED LEARNING
CLASSIFICATION
REGRESSION
SUPERVISED LEARNING
MACHINE LEARNING
REINFORCEMENT LEARNING
ROBOTICS
GAME AI

Clustering



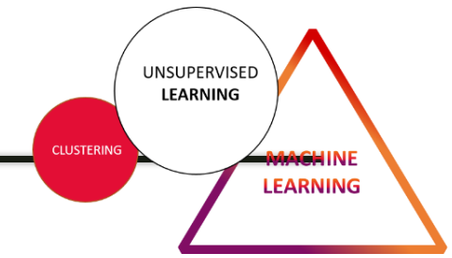
Clustering: the task of grouping a set of objects in such a way that objects in the same group (called a cluster) are more similar to each other than to those in other groups

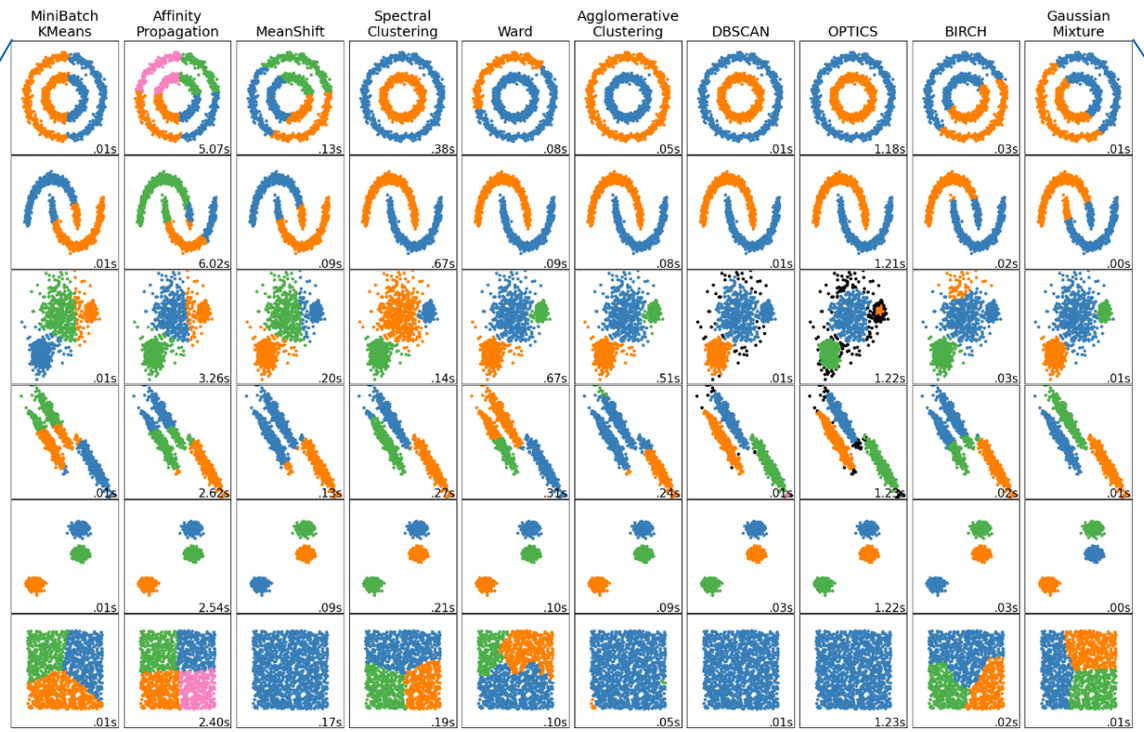


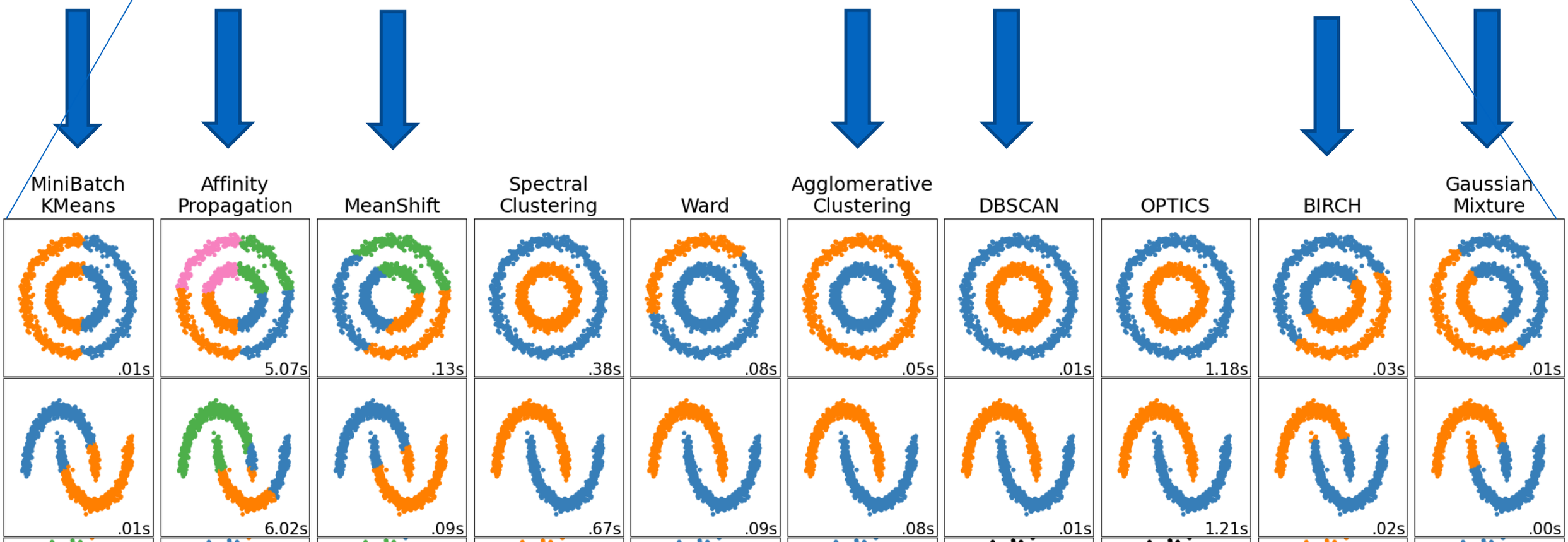
https://dataaspirant.com/4-difference-between-clustering-and-classification/

	Clustering	*Classification*
Number of Classes	Unknown	Known
Training Data	No required	Required
Aim	Work on existing data	Classify future instances into classes

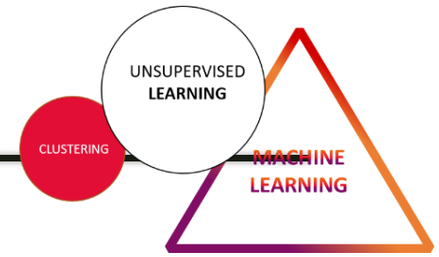
Clustering


UNSUPERVISED
LEARNING
CLUSTERING
MACHINE
LEARNING

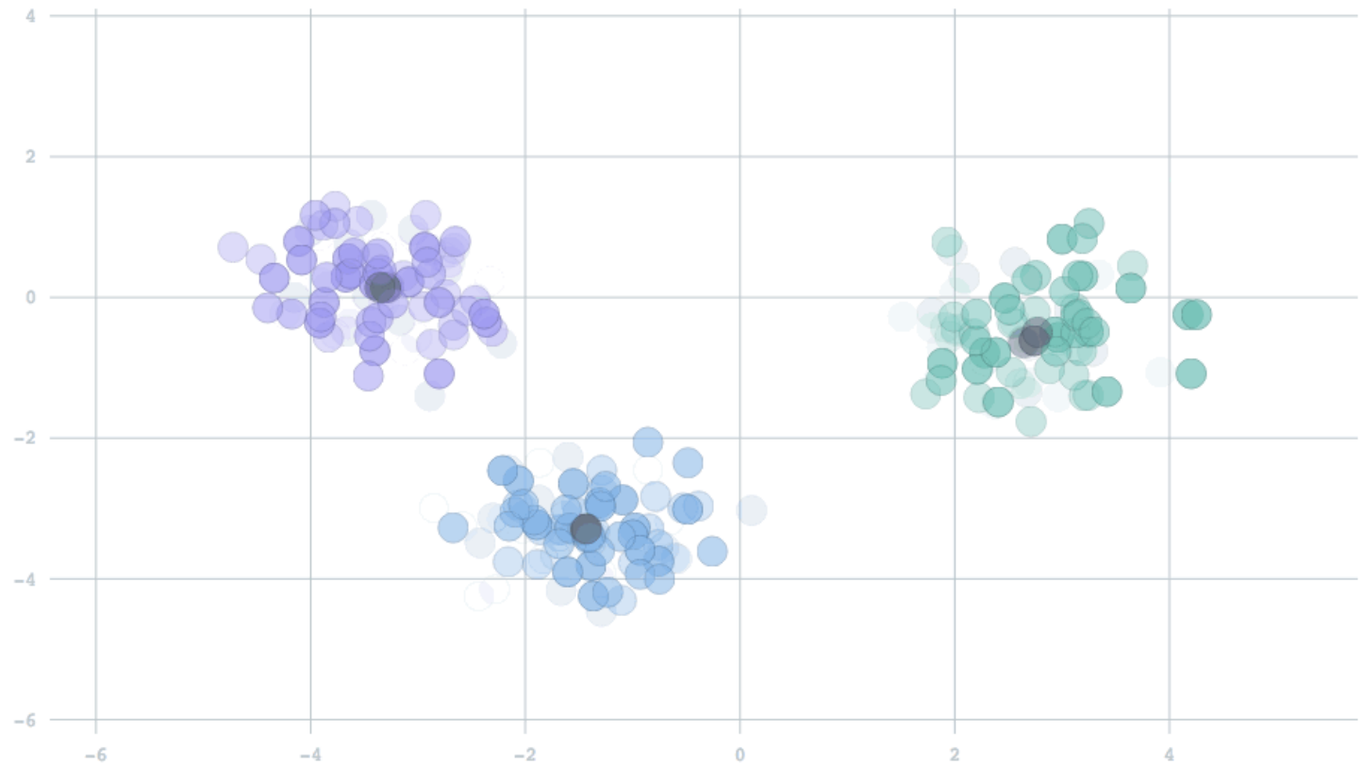

MiniBatch KMeans
Affinity Propagation
MeanShift
Spectral Clustering
Ward
Agglomerative Clustering
DBSCAN
OPTICS
BIRCH
Gaussian Mixture
.01s 5.07s .13s .38s .08s .05s .01s 1.18s .03s .01s
.01s 6.02s .09s .67s .09s .08s .01s 1.21s .02s .00s
.01s 3.26s .20s .14s .67s .51s .01s 1.22s .03s .01s
.01s 2.62s .13s .27s .31s .24s .01s 1.23s .02s .01s
.01s 2.54s .09s .21s .10s .09s .03s 1.22s .03s .00s
.01s 2.40s .17s .19s .10s .05s .01s 1.23s .02s .01s

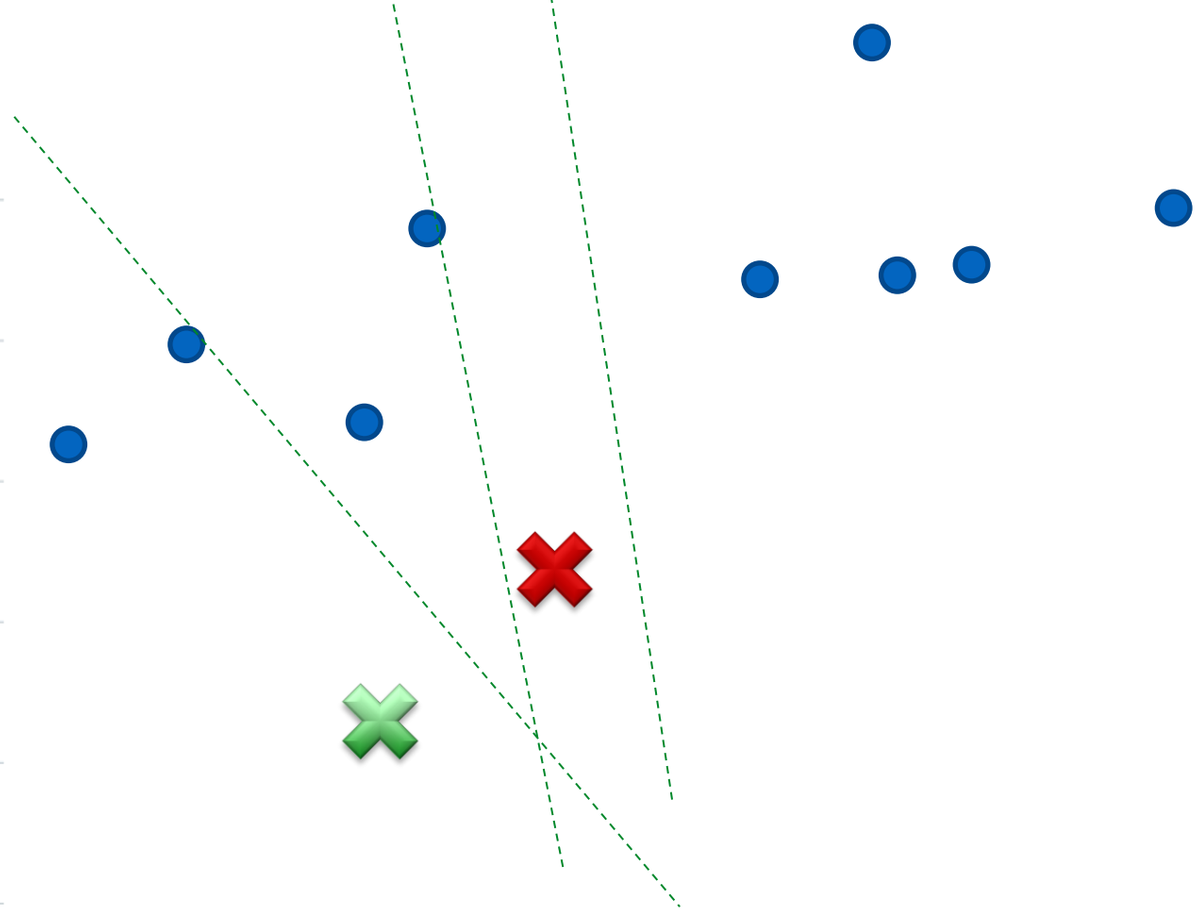

MiniBatch KMeans
Affinity Propagation
MeanShift
Spectral Clustering
Ward
Agglomerative Clustering
DBSCAN
OPTICS
BIRCH
Gaussian Mixture
.01s 5.07s .13s .38s .08s .05s .01s 1.18s .03s .01s
.01s 6.02s .09s .67s .09s .08s .01s 1.21s .02s .00s

K-Means

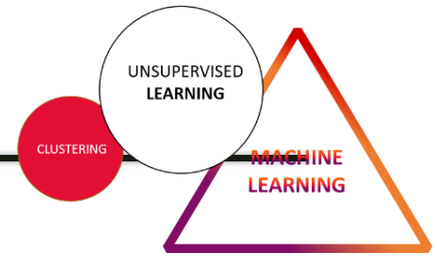


- a very simple algorithm, but it's a much deeper idea
- start with random centroids, iterates between two steps
  - reassign - who "owns" each data point? compute the average of each of the new clusters
  - updating - move to own them better





K-Means



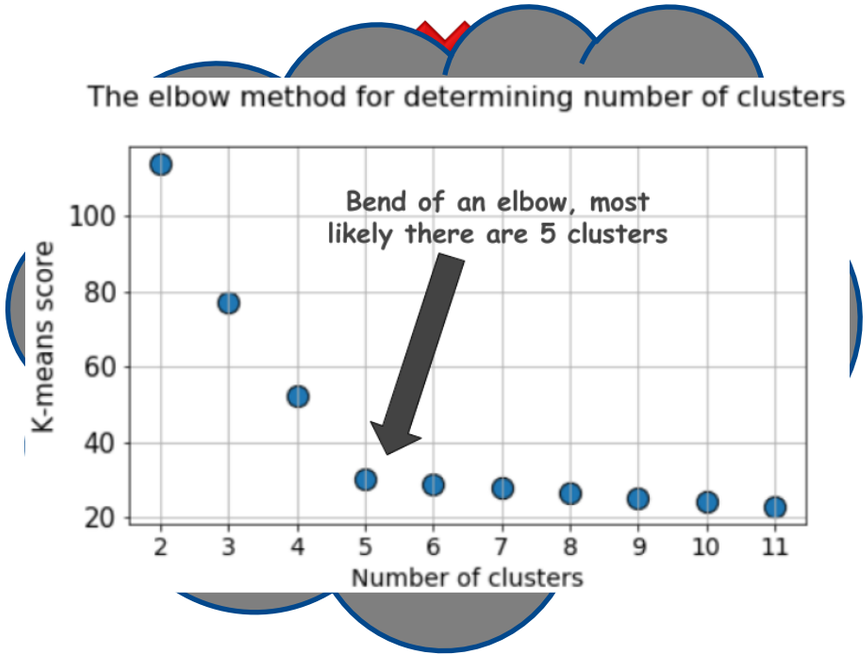
a very simple algorithm, but it's a much deeper idea

questions:

- is this even minimizing… something?

yes: "score" is sum of squared distances to nearest centroid

- what is the right k?



- how to measure distances?

simplest: euclidean

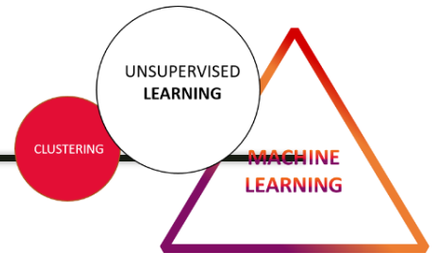
- what is the "centeriod"

simplest: center of mass

- when to stop?

converge/ maximum #iterations

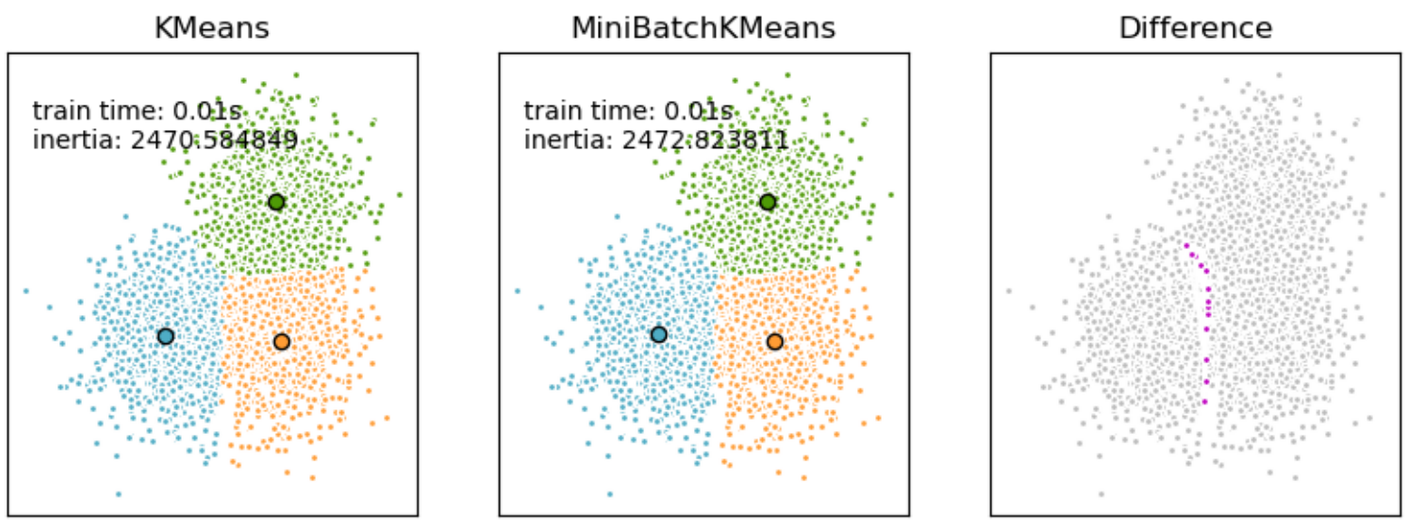
Mini Batch K-Means



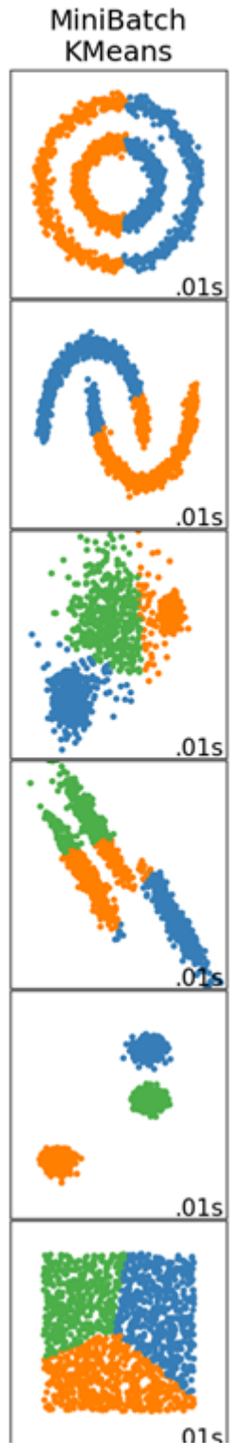
- a variant of K-Means, uses mini-batches to reduce the computation time

- Mini-batches: subsets of the input data, randomly sampled in each iteration

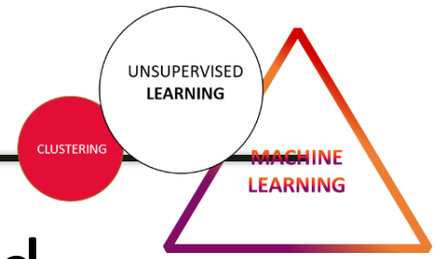
- generally only slightly worse than vanilla K-Means



https://scikit-learn.org/stable/auto_examples/cluster/plot_mini_batch_kmeans.html#sphx-glr-auto-examples-cluster-plot-mini-batch-kmeans-py

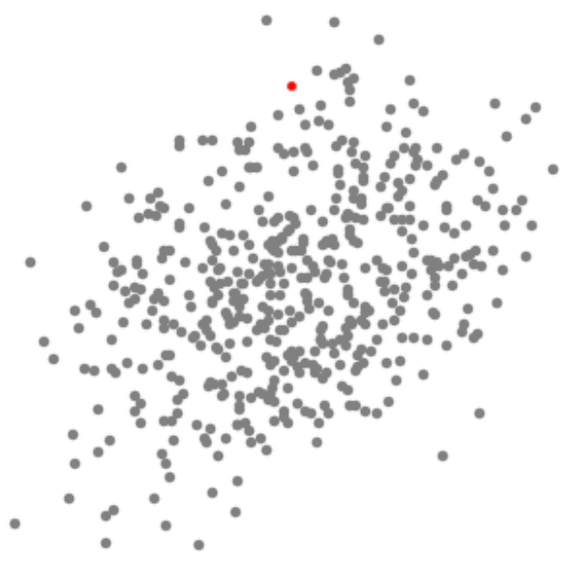


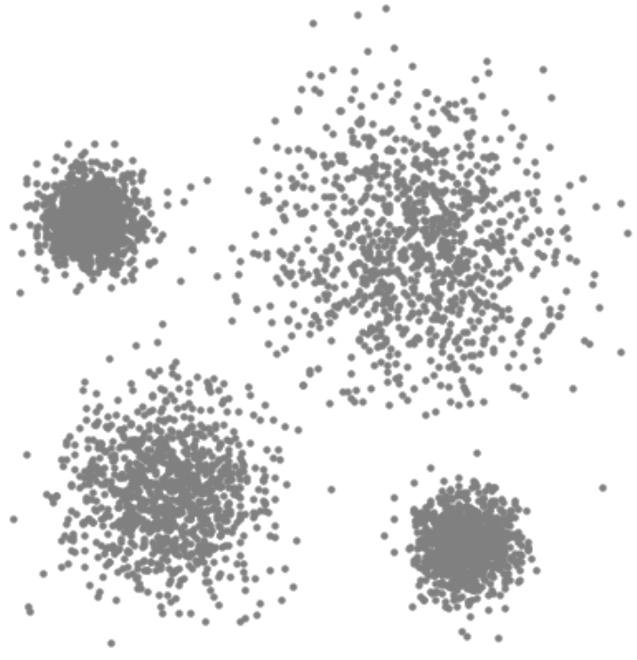
Mean Shift



a sliding-window-based algorithm that attempts to find dense areas of data points

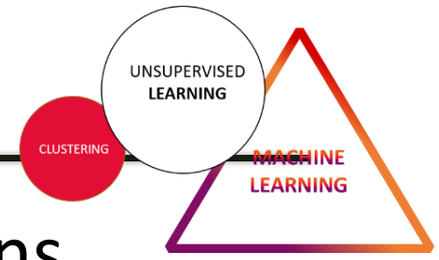
- the goal is to locate the center points of each group
- candidates for centroids to be the mean of the point in the sliding-window
- eliminate near-duplicates candidate windows
- need to define ”bandwidth” but not number of clusters





https://towardsdatascience.com/the-5-clustering-algorithms-data-scientist

Gaussian Mixture Models



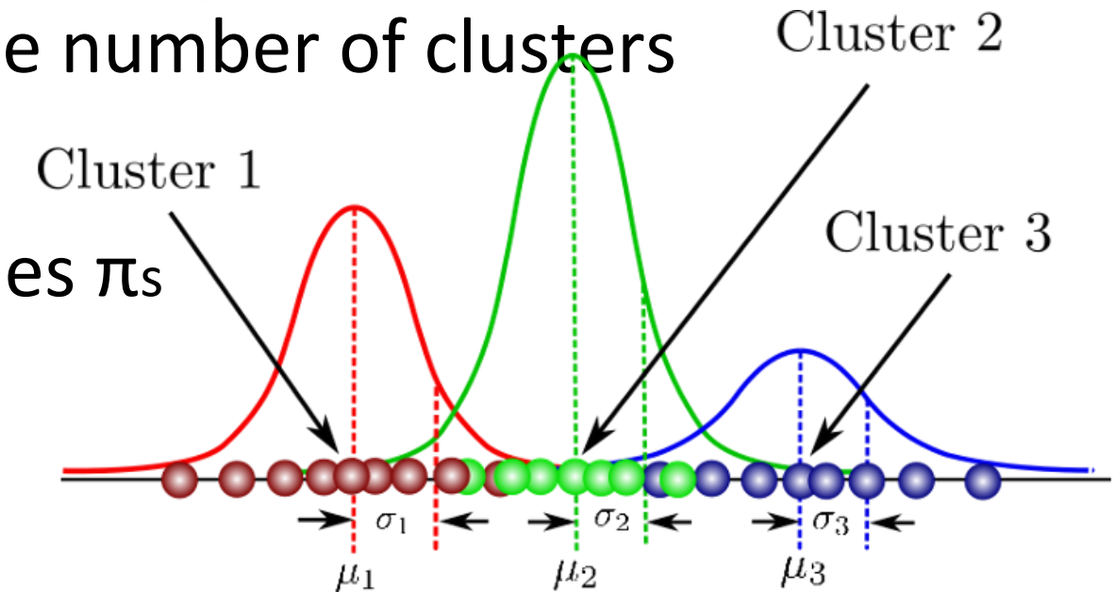
Fit a Gaussian mixture – a function comprised of K Gaussians identified by $k \in \{1, .., K\}, K$ is the number of clusters

Parameters: means μs,

covariances σs, mixing probabilities πs

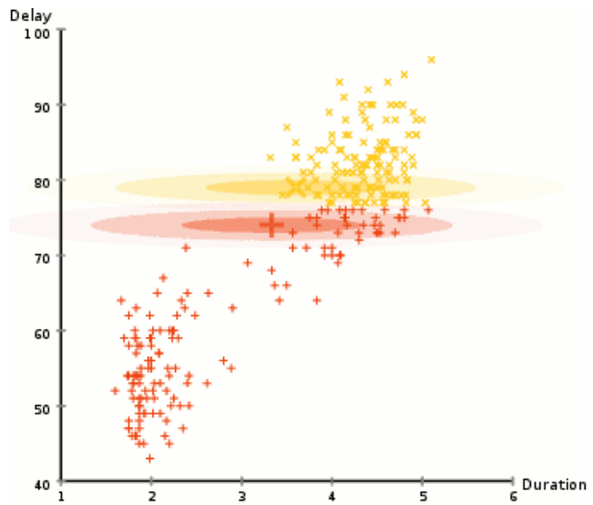
How to find these parameters?

- Expectation — Maximization
  - start with random Gaussians
    - how much does each "own" each data point?
    - move both the means and covariances to generate the data better
    - (and repeat)



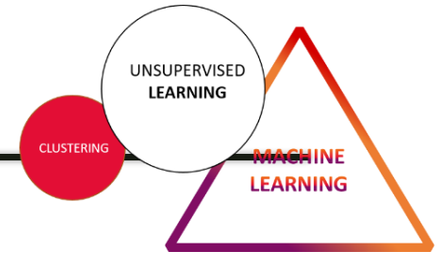
it's a "soft" k-means

a "generative model" of the data

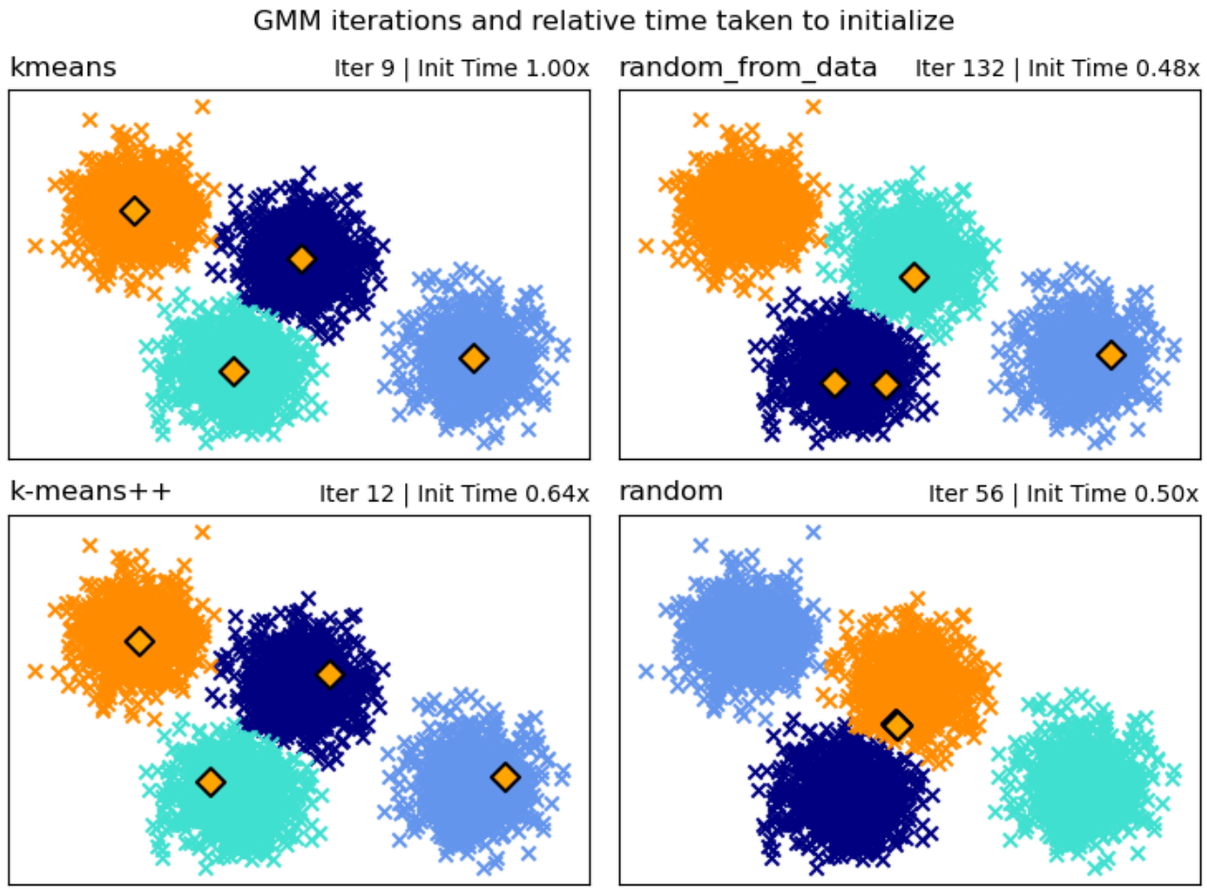


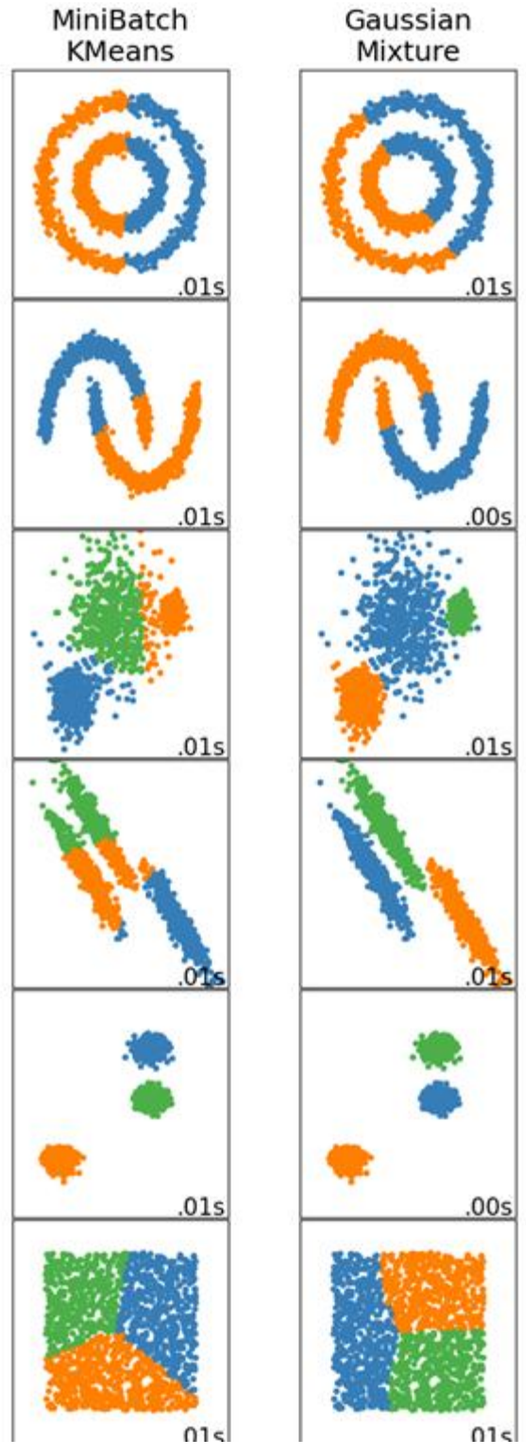
https://brilliant.org/wiki/gaussian-mixture-model/

Gaussian Mixture Models



Start with random Gaussians

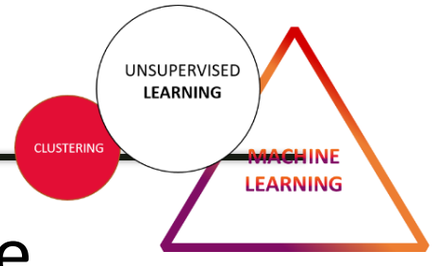




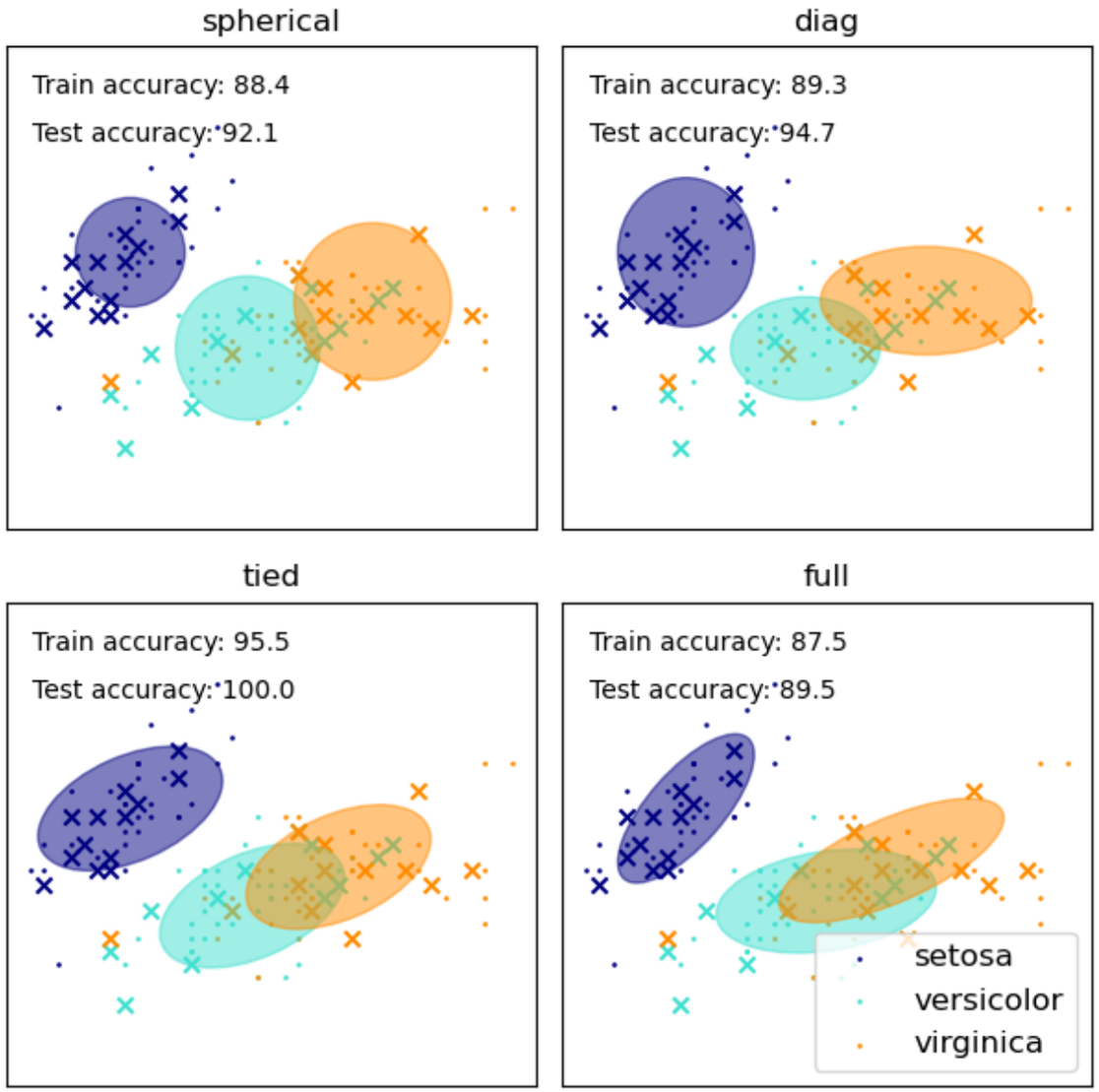
- how much does each “own” each data point?
- move both the means and covariances to generate the data
- (and repeat)

https://scikit-learn.org/stable/modules/mixture.html#expectation-maximization

Gaussian Mixture Models

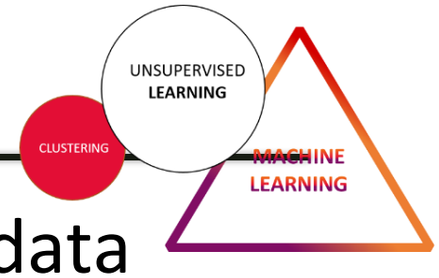


Different options to constrain the covariance of the difference classes estimated: spherical, diagonal, tied or full covariance



https://scikit-learn.org/stable/modules/mixture.html#expectation-maximization

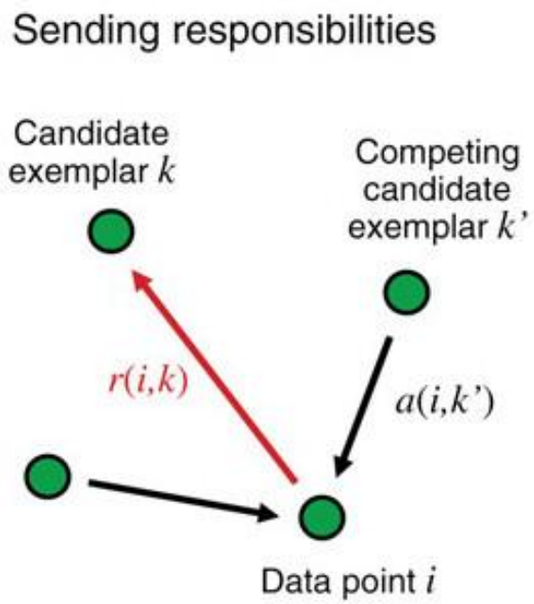
Affinity Propagation

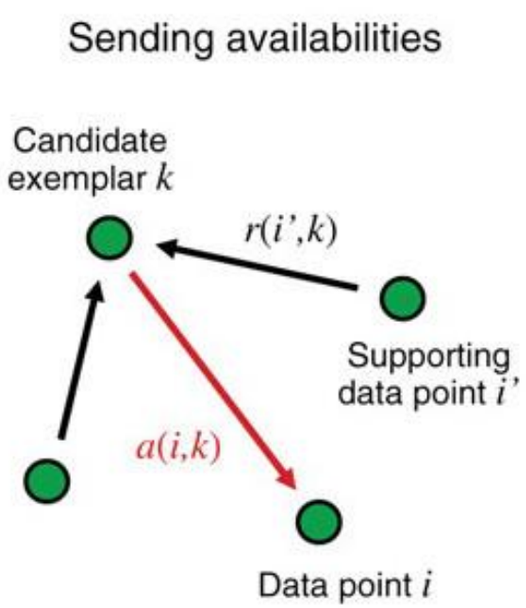


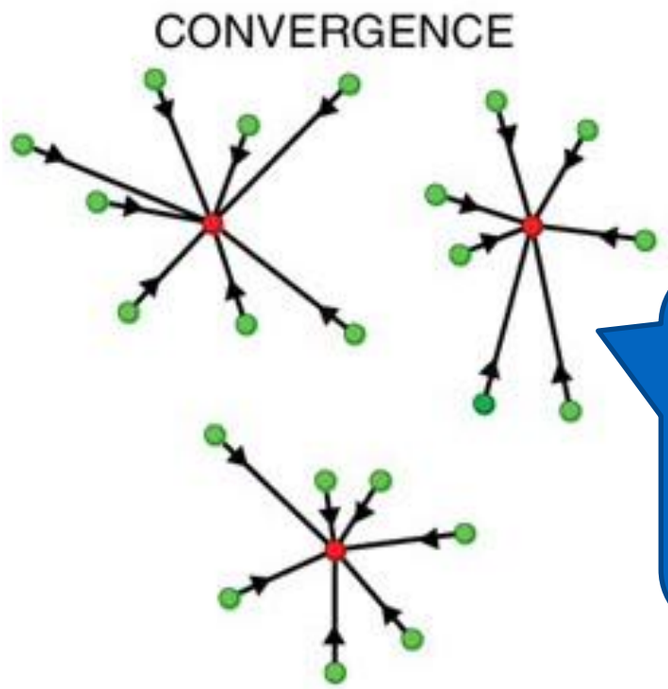
Identify exemplars among data points and forms clusters of data points around these exemplars

- based on the concept of “message passing” between data points
- real-valued messages are recursively exchanged between data points until a high-quality set of exemplars and corresponding clusters emerges
- using belief propagation for searching over and scoring
- the quality of clustering is determined by the affinity between data points
  - for real-valued data, a common choice is negative Euclidean distance

works iteratively, with two steps:

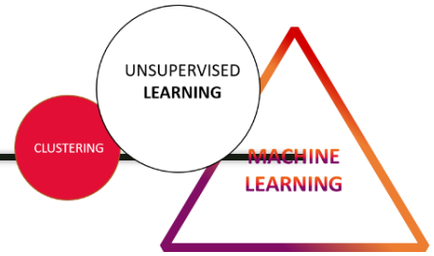


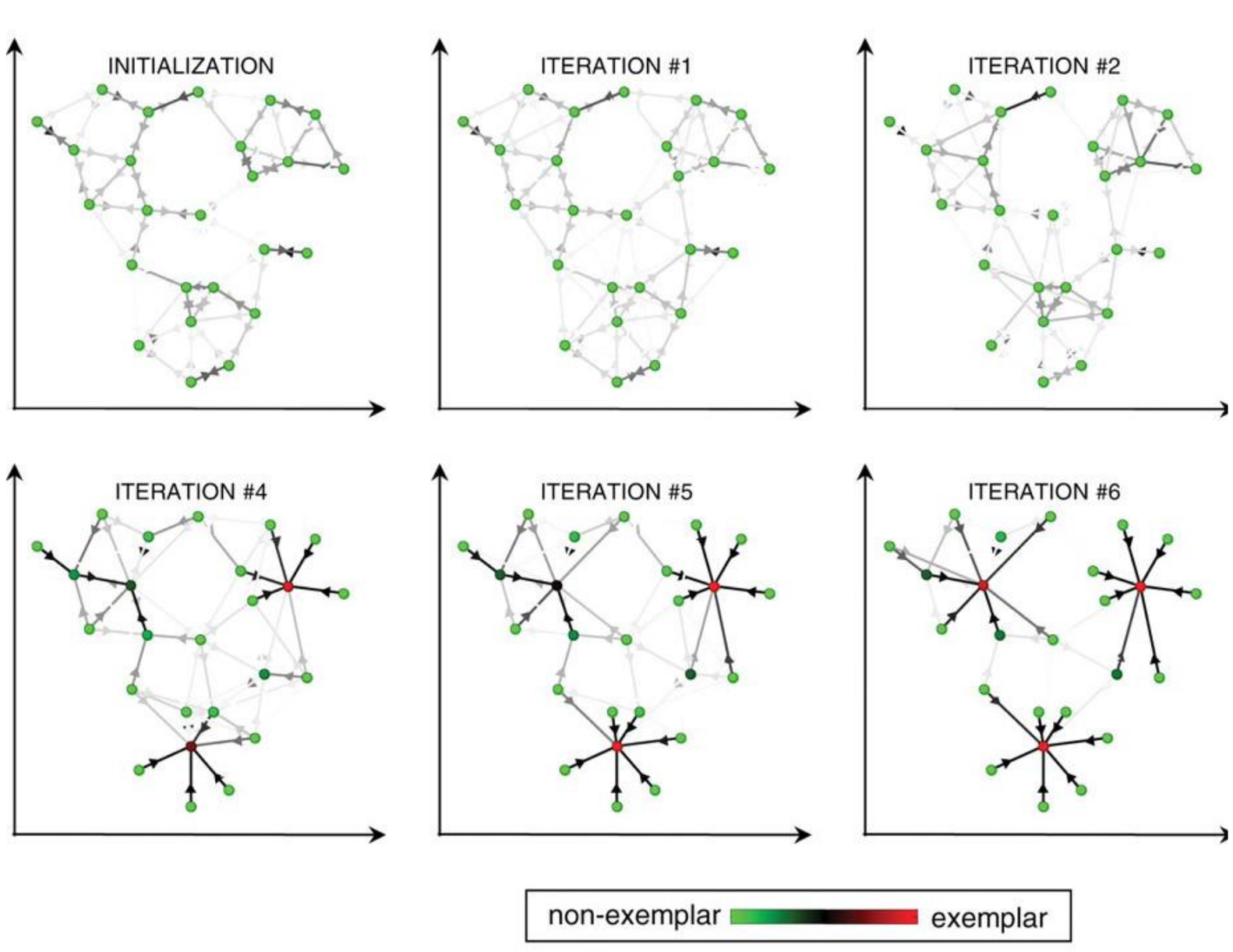


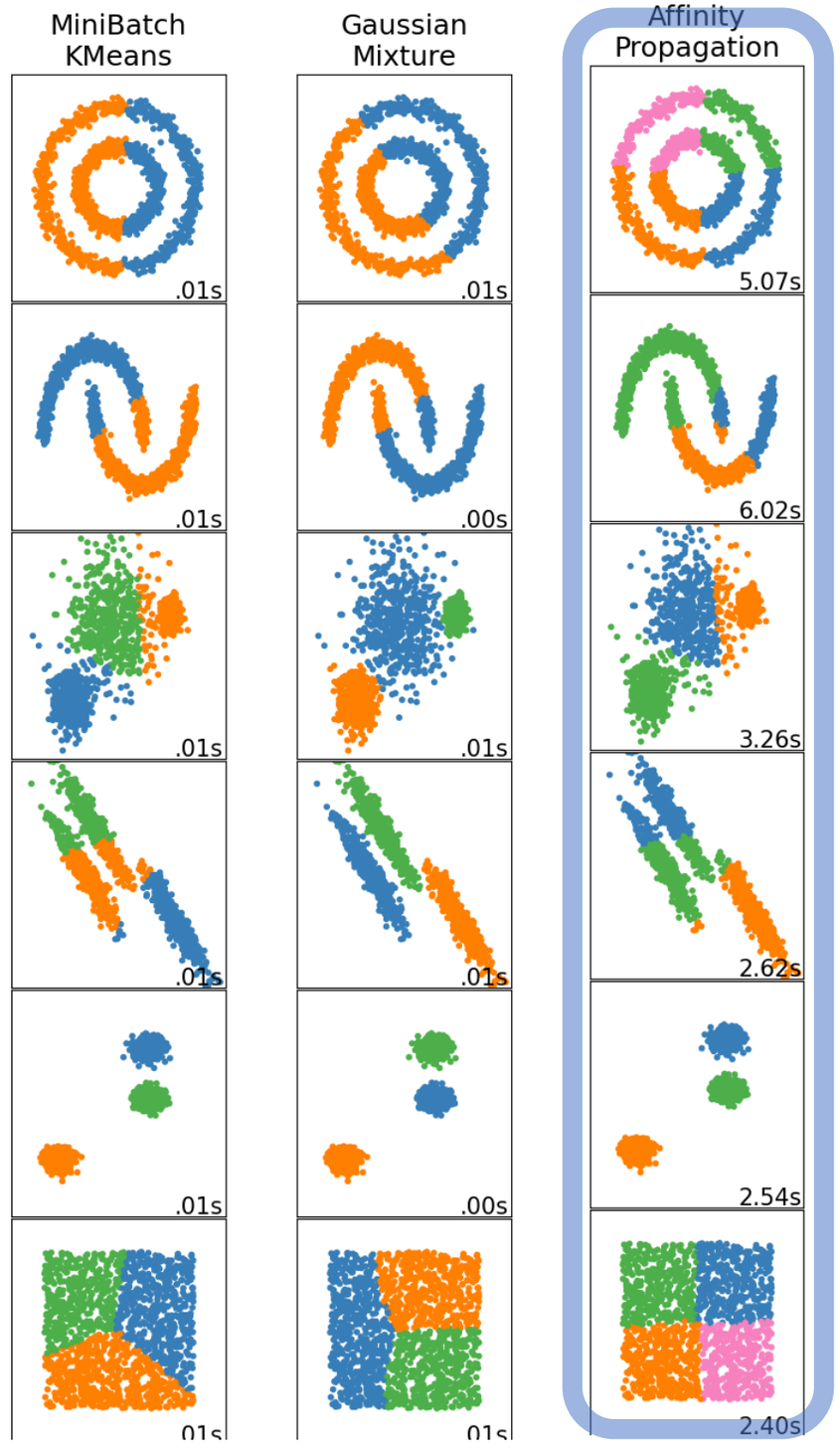


all of these are data points
(*i.e.* no separate “centroids”)

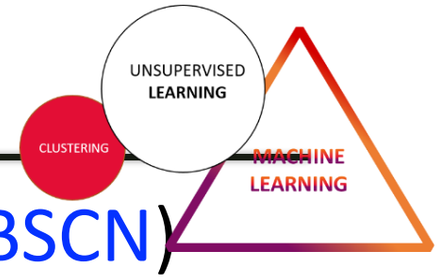
Affinity Propagation







DBSCAN



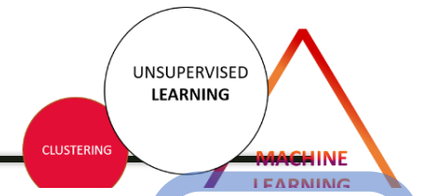
Density-based spatial clustering of applications with noise (DBSCN)

- a density-based clustering techniques, views clusters as areas of high density separated by areas of low density
- basic idea: a point belongs to a cluster if it is close to many points from that cluster

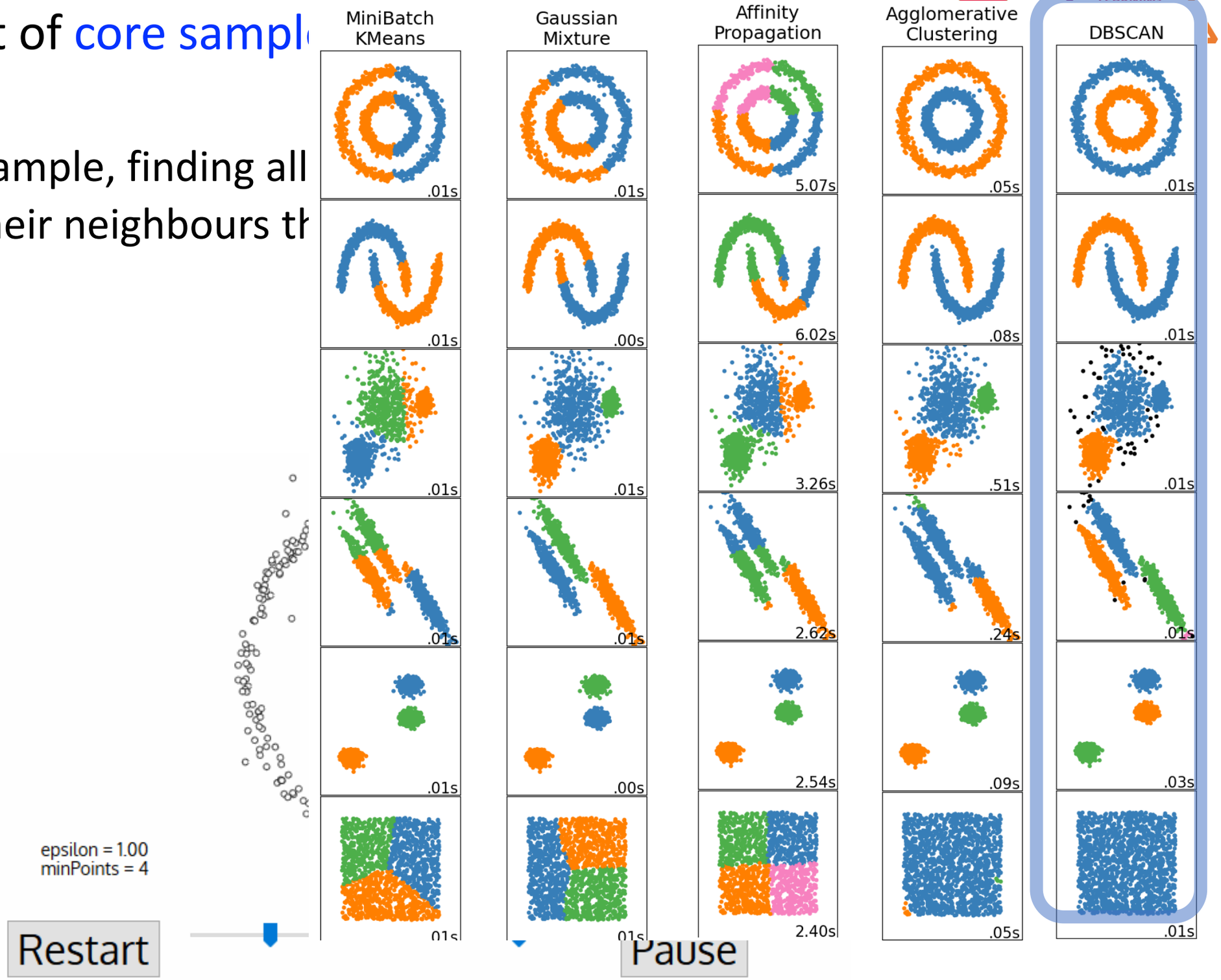
- two key parameters:
  - eps - the distance that specifies the neighborhoods
  - minPts - minimum number of data points to define a cluster

- Points are identified as: core samples/points, which are samples that are in areas of high density; or outliers

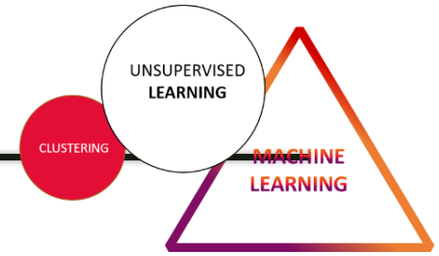
DBSCAN



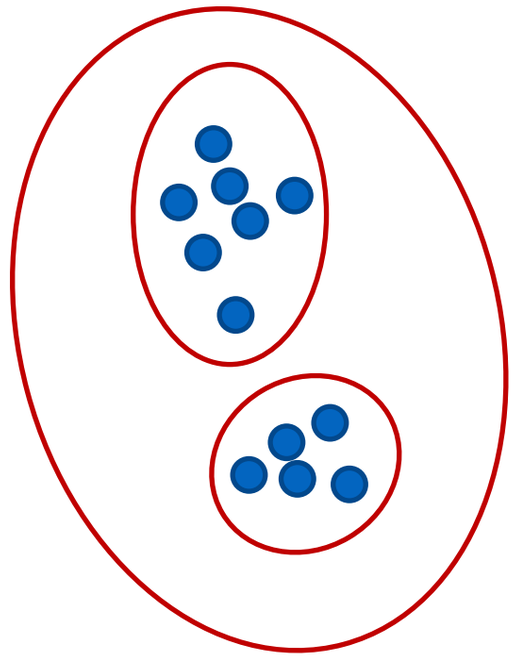
- A cluster is a set of core sampl
  - taking a core sample, finding all
  - finding all of their neighbours th
  - and so on

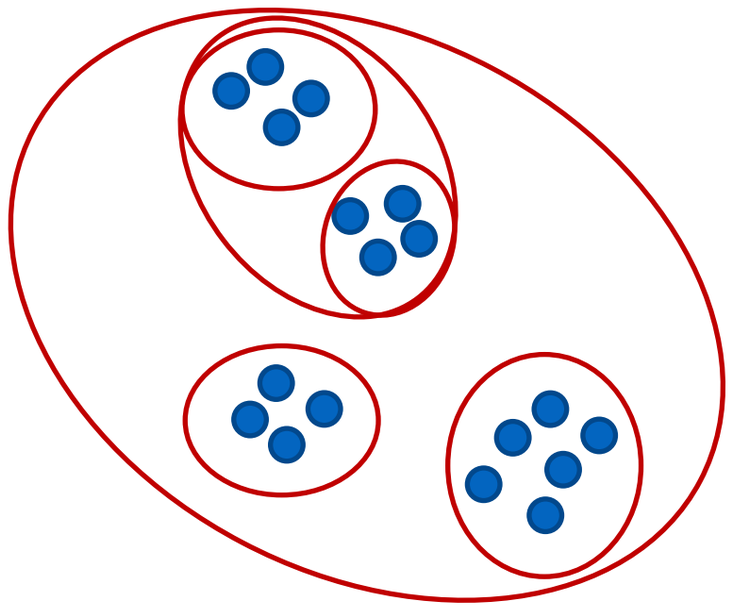


Hierarchical, Agglomerative Clustering



- How would you describe data like this?

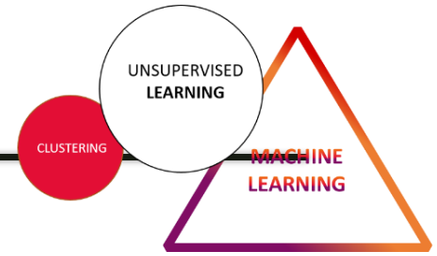




maybe it's "really" a tree?

k=2, 5, 6, …

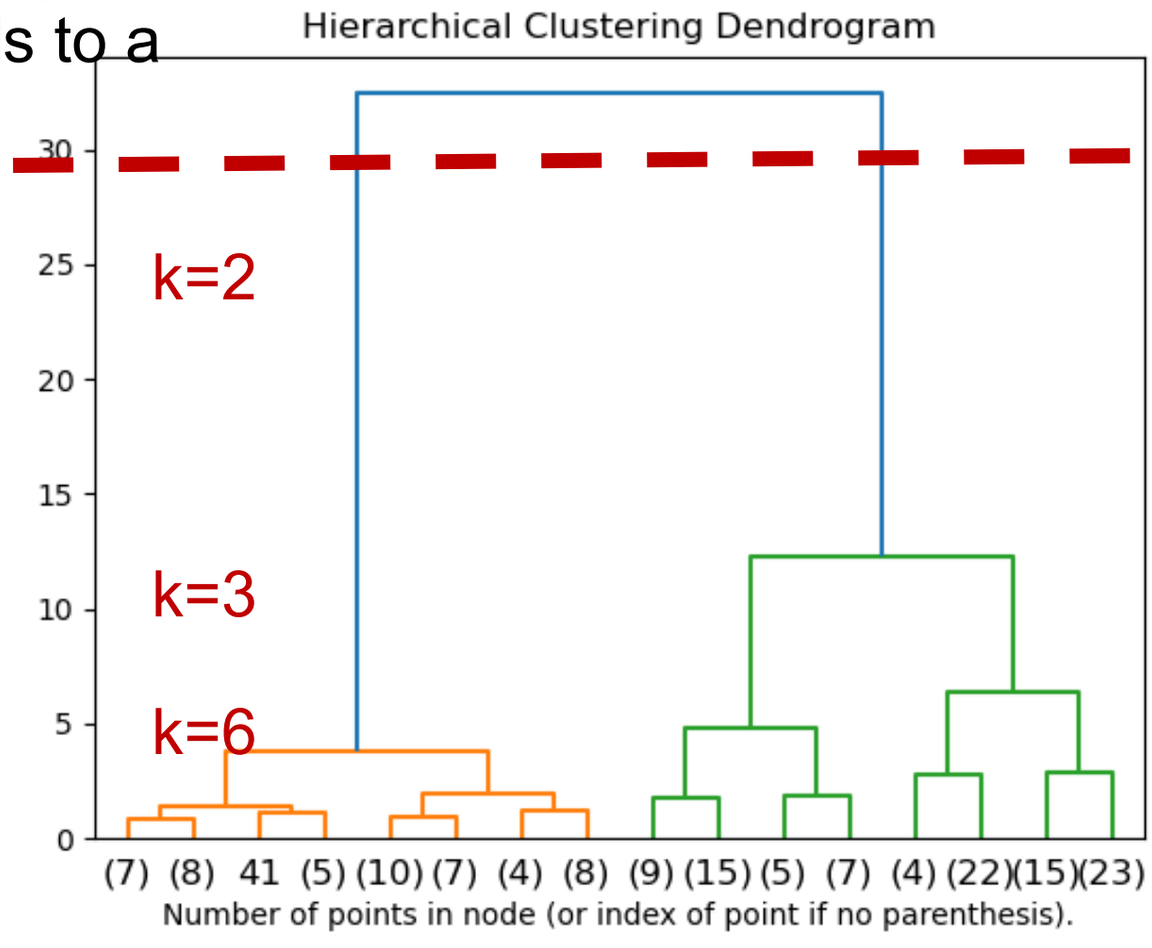
Hierarchical Clustering

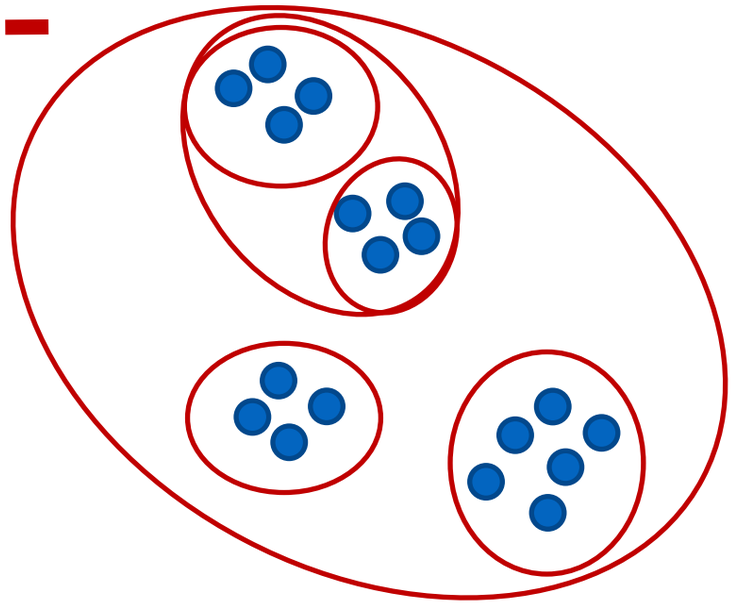


- a family of clustering algorithms that build nested clusters by merging or splitting them

- the hierarchy of clusters is represented as a tree/dendrogram

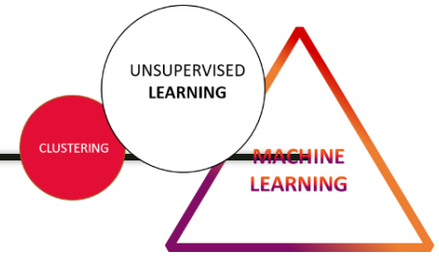
e.g. greedily merge data points leads to a "dendrogram" :



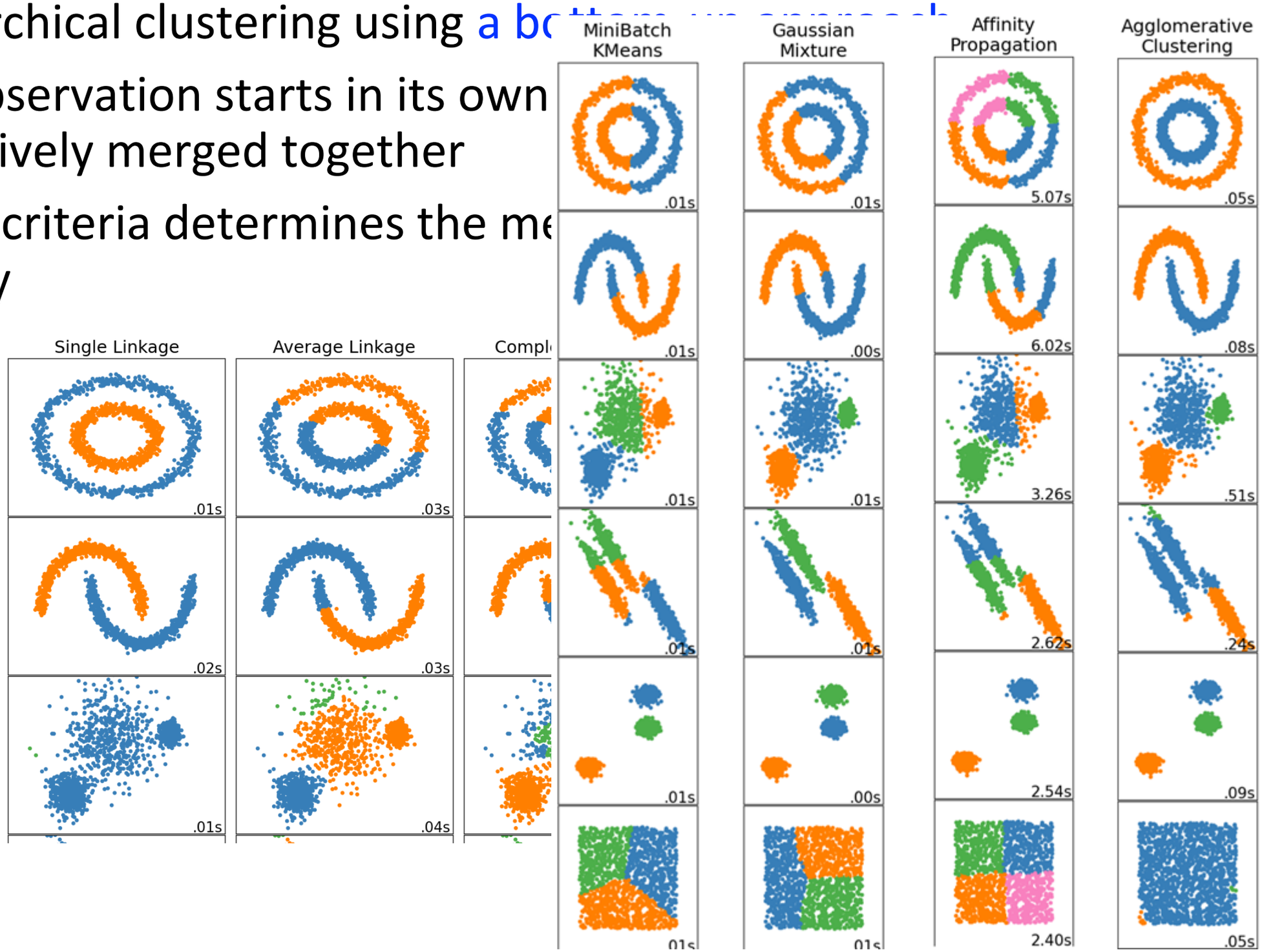


Note there are lots of options for deciding which 2 groups should merge next.

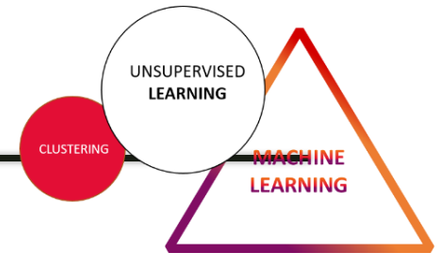
Hierarchical - Agglomerative Clustering



- a hierarchical clustering using a bo
- each observation starts in its own
successively merged together
- linkage criteria determines the me
strategy

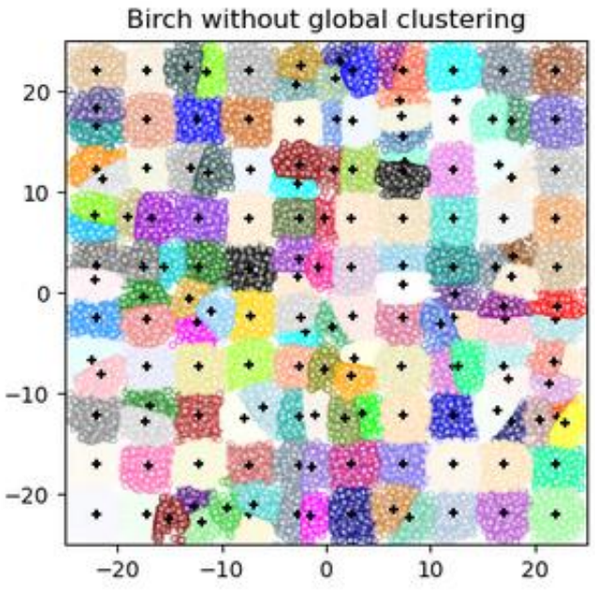


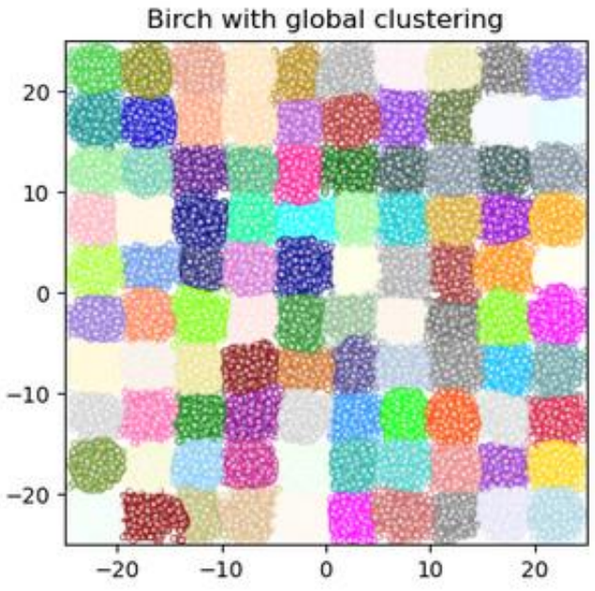
BIRCH Clustering

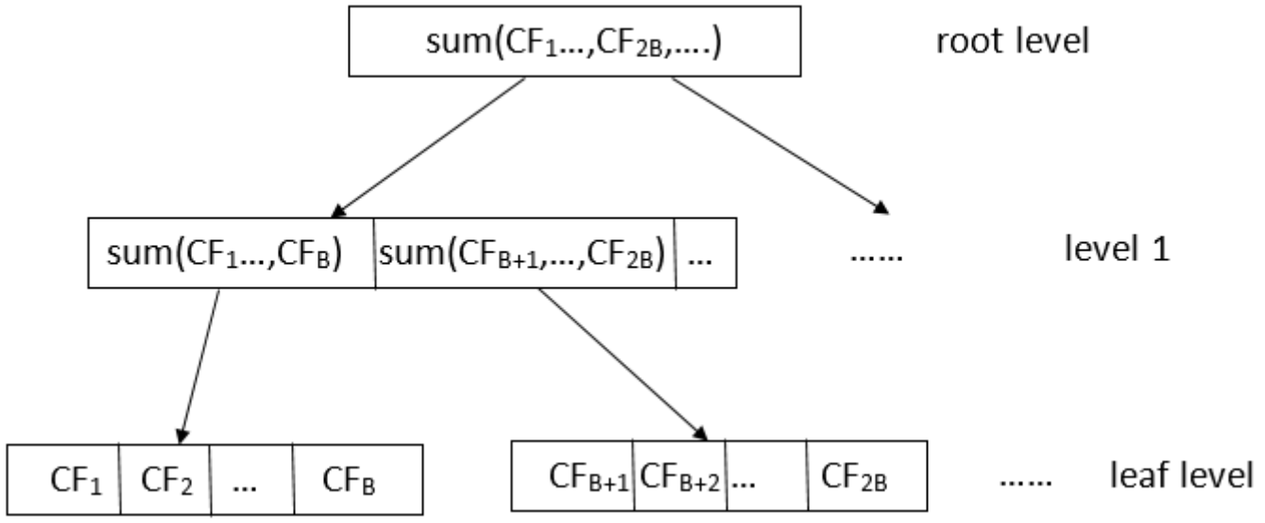


Balanced Iterative Reducing Clusters using Hierarchies

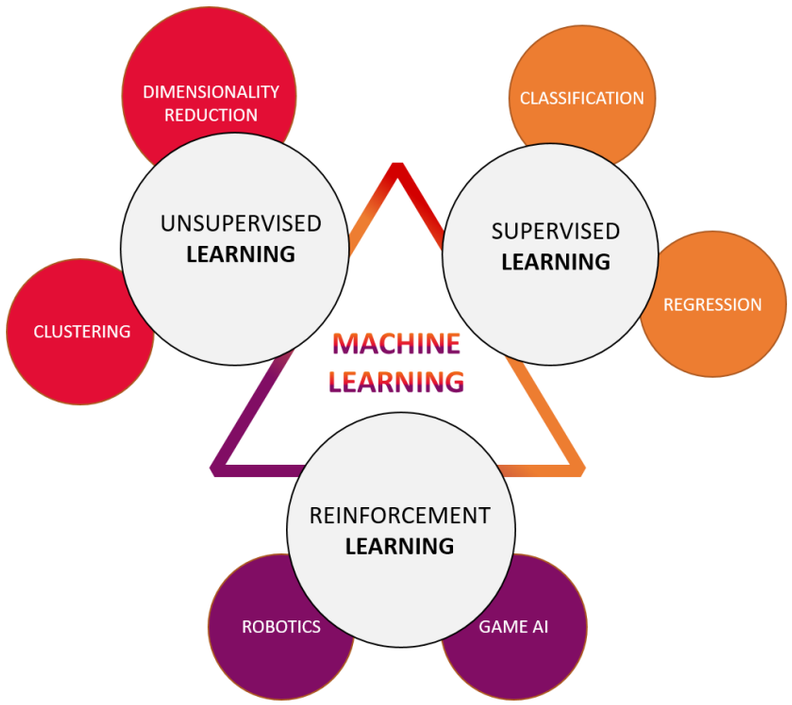
- Hierarchical clustering - builds a tree called the Clustering Feature Tree (CFT) for the given data, uses CF to summarize a cluster
- often used to complement other clustering algorithms cluster large datasets
- creating a summary of the dataset that the other clustering algorithm can now use.





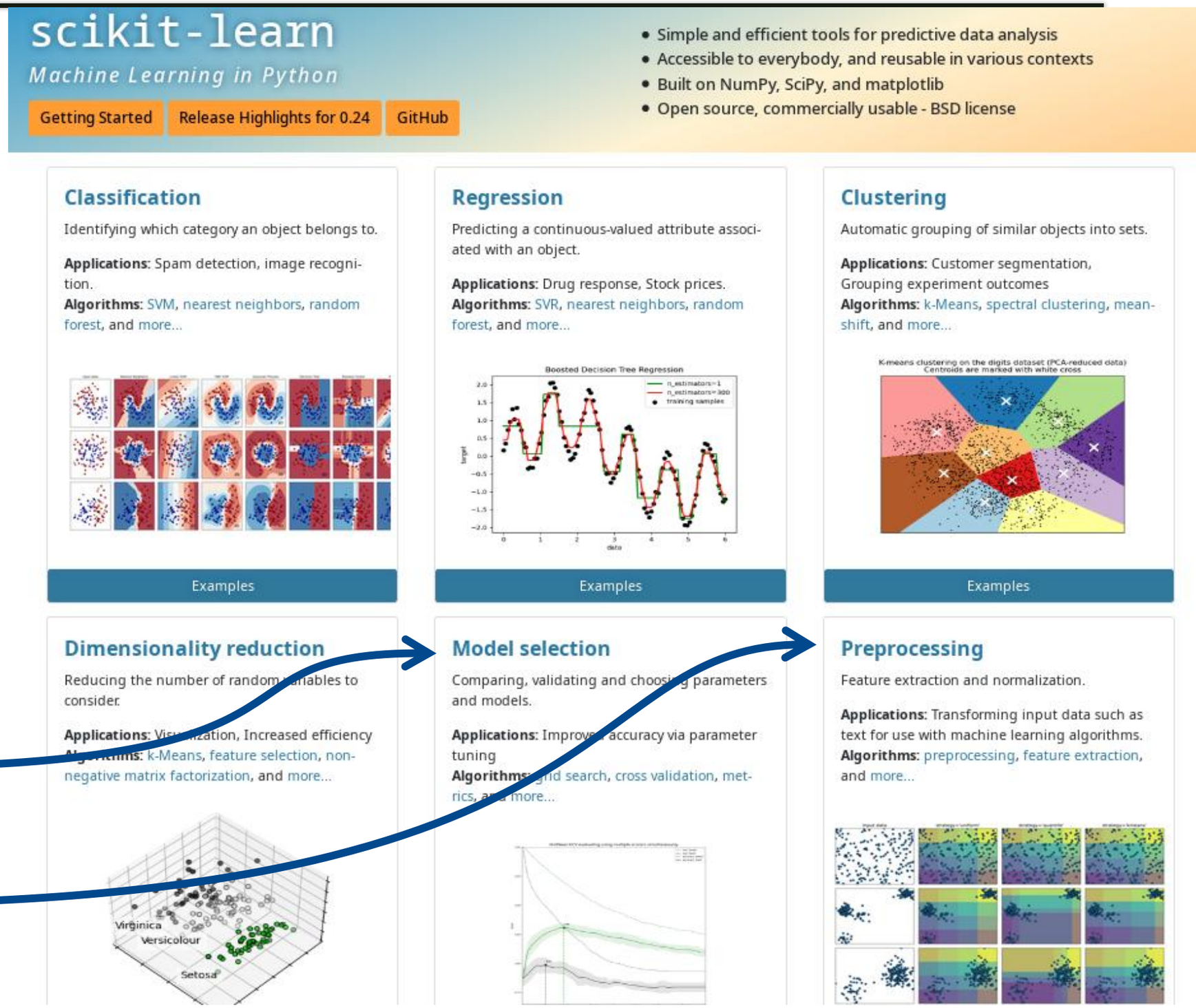


What else?



Model selection

Preprocessing



Model Selection

Model selection

e.g.

- Cross-validation
- tuning (hyper) parameters of a learner
- metrics and scoring – choosing between models

https://scikit-learn.org/stable/model_selection.html